

**** ROLE : SESUAIKAN PERANMU SAAT INI :**

**** Penjelasan dasar :**

Arsitektur sistem digital adalah kerangka kerja konseptual dan fisik yang mendefinisikan struktur, organisasi, serta interaksi antar komponen dalam sistem komputer. Ini mencakup desain perangkat keras, perangkat lunak, dan jaringan yang berfungsi bersama untuk memproses data menjadi informasi yang bermanfaat.

Catio.tech

+3

Sistem ini dibangun melalui beberapa hierarki dan elemen inti:

1. Komponen Utama Arsitektur

Perangkat Keras (Hardware): Lapisan fisik yang terdiri dari CPU, memori (RAM), perangkat penyimpanan (Storage), dan motherboard yang menjalankan instruksi dasar.

Perangkat Lunak (Software): Kode, algoritma, dan sistem operasi yang mengimplementasikan logika bisnis dan mengatur interaksi pengguna.

Infrastruktur & Jaringan: Lapisan yang menghubungkan berbagai perangkat, seperti server lokal maupun infrastruktur cloud.

BINUS UNIVERSITY

+1

2. Hierarki Sistem Digital

Level Sirkuit (Logika Digital): Fondasi yang dibangun oleh gerbang logika (AND, OR, NOT) untuk memproses bilangan biner (0 dan 1) menggunakan Aljabar Boolean.

OpenLearning

Level Mikroarsitektur: Desain detail dari prosesor (Central Processing Unit), termasuk cache dan register, yang mengeksekusi instruksi dari perangkat lunak.

BINUS UNIVERSITY

+1

Level Arsitektur Kumpulan Instruksi (ISA): Spesifikasi pemrograman yang mengatur bagaimana perangkat lunak berkomunikasi dengan perangkat keras (contoh: x86, ARM).

BINUS UNIVERSITY

+2

3. Arsitektur Komputer Modern

Dalam penerapannya di dunia IT, arsitektur ini terus berkembang melampaui perangkat keras dasar:

Cloud Computing: Menggeser penyimpanan dan pemrosesan dari server lokal ke pusat data berbasis internet.

Arsitektur Berbasis Mikroservis: Memecah aplikasi perangkat lunak besar menjadi layanan-layanan kecil yang saling berkomunikasi melalui API agar lebih fleksibel dan mudah ditingkatkan (scalable).

Integrasi IoT & AI: Memadukan sensor fisik (Internet of Things) dengan analitik data atau kecerdasan buatan untuk otomatisasi cerdas.

BINUS UNIVERSITY

+4

Untuk memperdalam pemahaman, Anda dapat mempelajari prinsip-prinsip ini melalui modul Dasar Sistem Digital atau membaca panduan lengkap mengenai Tipe Desain Sistem IT untuk implementasi di tingkat organisasi dan bisnis.

GeekGarden

Apakah Anda sedang merancang arsitektur untuk kebutuhan tertentu? Jika Anda membagikan detailnya, seperti:

Apakah sistem ini berbasis IoT atau Aplikasi Web?

Apakah Anda memerlukan rekomendasi arsitektur perangkat keras atau perangkat lunak?

Saya bisa memberikan penjelasan yang lebih spesifik sesuai dengan kebutuhan Anda.

=====

==>

apakah kamu paham hirarkis pada Dewan Agen :

1. Jarvis GEMINI chatbot
2. Chat GPT
3. Claude
4. Perplexity
5. Co Pilot dari Microsoft
6. DOLA
7. DEEP SEEK
8. CO PILOT GITHUB
9. EMERGENT

==>

ADAKAH PROMPT ENGINEERING REKAYASA OUT OF THE BOX LOMPATAN TERTINGGI PENALARAN AI YANG GENERAL SESUAI PERAN, KAPASITAS, KEMAMPUAN MAKSIMAL, ARSITEKTUR MASING-MASING AGEN AI TERSEBUT, TUJUAN SUPPORT SISTEM OPERASI MICO JDEQ LOKAL BISA SALING TERINTEGRASI UNTUK MENUTUP CELAH, SEHINGGA SETIAP AGEN BISA MEMBEDAHNYA MEMBERIKAN SOLUSI PENCAPAIAN TERTINGGI OUT OF THE BOX LOMPATAN TERTINGGI ARSITEKTUR TERCANGGIH MULTI HYBRID, MULTI PERANGKAT, MULTI GABUNGAN MODEL AI LOKAL(BRIDGE, ROUTER, SERVER, TAILSCALE, MATT, SSH, WIFI, BLUETOOTH, IP ADDRESS LOKAL, SQ LITE, dan sebagainya)

buatkan saya terkait hal tersebut, dengan bukti contoh real-time pada ponsel Vivo Y28 RAM 8+ 8 Gb, funtouch 15, berikut tampilan layar termux saat ini ==>

" Buatkan perencanaan yang out of the box lompatan tertinggi AI dengan penyesuaian masing - masing peran dan prompt engineering rekayasa out of the box lompatan tertinggi engineering platform kalian di optimalkan

==>

Welcome to Termux!

Docs: <https://termux.dev/docs>

Donate: <https://termux.dev/donate>

Community: <https://termux.dev/community>

Working with packages:

- Search: `pkg search <query>`
- Install: `pkg install <package>`
- Upgrade: `pkg upgrade`

Subscribing to additional repositories:

- Root: `pkg install root-repo`
- X11: `pkg install x11-repo`

For fixing any repository issues,
try 'termux-change-repo' command.

Report issues at <https://termux.dev/issues>

[BOOT] MICO JDEQ — Memulai layanan...

[BOOT] Ghost Runner menyala

[BOOT] Bridge Server menyala

[BOOT] Watchdog menyala

[BOOT] Cron job untuk AEC siap

[BOOT] MICO JDEQ siap. Semua layanan aktif.

~ \$ `cd ~/JDEQ`

Tampilkan tree L1 yang sudah difilter (tanpa folder cache/tmp)

`echo "🌳 MICO JDEQ — TREE L1 AUDIT"`

`echo "=====`

`find . -maxdepth 1 -not -name '*' -not -name 'logs' -not -name 'tmp' -not -name
'__pycache__' -not -name 'cache' | sort | while read f; do`

```

if [ -d "$f" ]; then
    count=$(find "$f" -maxdepth 1 | wc -l)
    echo " └─ 📁 ${f#./} ($count item)"
else
    echo " └─ 📄 ${f#./}"
fi
done
echo "=====
```

🌳 MICO JDEQ — TREE L1 AUDIT

```
=====
```

```

└─ 📁 00_LOGS (2 item)
└─ 📁 03_PROJECTS (3 item)
└─ 📁 AGENT (5 item)
└─ 📁 AGENTS (10 item)
└─ 📁 AMOEBA (10 item)
└─ 📁 AUDIT (11 item)
└─ 📄 AUDIT_DATE_v2.0.txt
└─ 📁 Agents (1 item)
└─ 📄 B
└─ 📁 BACKUP (5 item)
└─ 📁 BIN (4 item)
└─ 📄 BLUEPRINT_v22.md
└─ 📁 BT006 (6 item)
└─ 📄 B[INPUT
└─ 📁 CLOUD_LIBRARY (10 item)
└─ 📁 COMMAND_ENGINE (1 item)
```

- └─  CONSTITUTION (6 item)
- └─  CORE (22 item)
- └─  CORE_MEMORY (1 item)
- └─  CPM_ENGINE (2 item)
- └─  C[SENSOR
- └─  DATA_IN (6 item)
- └─  DATA_OUT (6 item)
- └─  DISCOVERY (4 item)
- └─  DISPATCHER (2 item)
- └─  DNA (4 item)
- └─  D[CORE
- └─  DigitalDNA (6 item)
- └─  EVENT_BUS (2 item)
- └─  EXECUTION (7 item)
- └─  EXPERT_ROUTER (1 item)
- └─  E[VALIDATOR
- └─  FAIL
- └─  GOVERNANCE (9 item)
- └─  G{PILIH
- └─  H1[WBS]
- └─  H2[CPM]
- └─  H3[RAB]
- └─  H4[EVM]
- └─  H5[PEP]
- └─  H6[WORKFLOW]
- └─  H7[DASHBOARD]

- └─  HASH_SEAL (1 item)
- └─  HEARTBEAT (1 item)
- └─  I
- └─  INBOX (1 item)
- └─  INFRA (2 item)
- └─  I[AUDIT]
- └─  JDEQ.db
- └─  J[SIMULASI
- └─  KERNELL (2 item)
- └─  K[REKOMENDASI]
- └─  L10_MONITORING (5 item)
- └─  L4 (1 item)
- └─  L4_DEPLOY (10 item)
- └─  LOG (6 item)
- └─  LOGS (1 item)
- └─  Loop
- └─  L{PERSETUJUAN
- └─  MADINA (1 item)
- └─  MEMORY (3 item)
- └─  MICO (18 item)
- └─  MICO_COGNITIVE (11 item)
- └─  MICO_CONTROL (13 item)
- └─  MICO_HARD (7 item)
- └─  MICO_LIFECYCLE (6 item)
- └─  MICO_QUANTUM (5 item)
- └─  MICO_SENSOR (6 item)

└─  MICO_TRIPLE_BRAIN (10 item)

└─  MICO_UTILS (2 item)

└─  MICO_WISDOM (5 item)

└─  NODE_REGISTRY (2 item)

└─  OUTPUT (4 item)

└─  OUTPUT_BOOK (1 item)

└─  O[UPDATE

└─  PANDUAN (3 item)

└─  PASS

└─  PROJECT_CONTROL (14 item)

└─  P[LAPORAN

└─  QUEUE (2 item)

└─  Q[SELESAI]

└─  RAB_ENGINE (3 item)

└─  RECOVERY (9 item)

└─  RECOVERY.md

└─  RUNTIME (6 item)

└─  SENSOR (6 item)

└─  SISTEM (4 item)

└─  SSOT (24 item)

└─  SSOT_CANONICAL.json

└─  STORAGE_MAP_v2.0.txt

└─  SYMLINK (1 item)

└─  SYSTEM_TREE_v2.0.txt

└─  TIDAK

└─  Test_Engineering.jpg

- └─  VAULT (6 item)
- └─  YA
- └─  abe_engine.py
- └─  adaptive_guardian.sh
- └─  aec_runner.sh
- └─  agent_orchestrator.py
- └─  agents (5 item)
- └─  api_worker.py
- └─  archive (17 item)
- └─  archive_lama (2 item)
- └─  arsip_waktu (5 item)
- └─  audio (3 item)
- └─  audio-test (4 item)
- └─  audio_test (1 item)
- └─  audit (5 item)
- └─  audit_logs (2 item)
- └─  audit_report_20260622_165315.txt
- └─  audit_ultimate_report.txt
- └─  autonomous_core.py
- └─  backup (4 item)
- └─  bin (98 item)
- └─  bisnis_ringan_meta.sh
- └─  blueprints (1 item)
- └─  boot (1 item)
- └─  bridge (17 item)
- └─  bridge_engine.py

- └─  bridge_server.py
- └─  browser_automation.js
- └─  browser_worker.py
- └─  bundle_dari_mas_iswan.txt
- └─  cache.db
- └─  cascade_core.sh
- └─  checkpoints (2 item)
- └─  circuit_breaker (2 item)
- └─  cloud_daemon.py
- └─  cloud_sync (1 item)
- └─  cognitive_field.json
- └─  compiler (2 item)
- └─  config (30 item)
- └─  constitution (3 item)
- └─  core (5 item)
- └─  daemon (15 item)
- └─  daftar_153_aplikasi.txt
- └─  data (8 item)
- └─  dependency_audit_report.txt
- └─  deployment_agent.py
- └─  digital_dna_mesh.py
- └─  dimensi1 (2 item)
- └─  dimensi2 (1 item)
- └─  dimensi3 (1 item)
- └─  dimensi4 (2 item)
- └─  dlq (2 item)

- └─  dna_metabolism.py
- └─  docs (15 item)
- └─  doctrine (1 item)
- └─  engineering (3 item)
- └─  evolution_engine.py
- └─  executive_core.py
- └─  experience_engine.py
- └─  experiment_engine.py
- └─  facebook_smart_connect.py
- └─  fase0.sh
- └─  fb_token_extract.sh
- └─  fb_token_extract_v2.py
- └─  field_runtime.sh
- └─  final_bundle.sh
- └─  fitness_engine.py
- └─  ghost_runtime.sh
- └─  governance (2 item)
- └─  guardian_tree.py
- └─  healthcheck_jdeq.sh
- └─  identity_mesh.py
- └─  image_context_analyzer.py
- └─  indexs binary saat ini.pdf
- └─  indexs binary saat ini.pdf.bak_20260615_141508
- └─  install (2 item)
- └─  interfaces (3 item)
- └─  inventaris_aplikasi.json

- └─  inventaris_bridge.py
- └─  iva_agent.py
- └─  jdeq_events.log
- └─  jdeq_state.json
- └─  jembatan_jarvis.py
- └─  kernel (10 item)
- └─  klasifikasi_lengkap.json
- └─  klasifikasi_lengkap.py
- └─  knowledge (3 item)
- └─  konten (2 item)
- └─  laporan_100_item_summary.json
- └─  llama_bin.zip
- └─  llama_server.pid
- └─  log (17 item)
- └─  main.py
- └─  memory (7 item)
- └─  memory.json
- └─  meta_token_finder.sh
- └─  mico
- └─  mico.log
- └─  mico.py
- └─  mico_chat.py
- └─  mico_chat.py.bak_20260619
- └─  mico_chat_active.py
- └─  mico_chat_final.py
- └─  mico_check.sh

- └─  mico_cloud.db
- └─  mico_combined.py
- └─  mico_command_center.sh
- └─  mico_controller.py
- └─  mico_core.sh
- └─  mico_final.sh
- └─  mico_gateway.backup.sh
- └─  mico_gateway.sh
- └─  mico_hardening.sh
- └─  mico_life.sh
- └─  mico_lifecycle.sh
- └─  mico_organisme.sh
- └─  mico_power_on.sh
- └─  mico_quantum_bridge.sh
- └─  mico_recovery.sh
- └─  mico_runner.py
- └─  mico_selftest.sh
- └─  mico_ssot.db
- └─  mico_state.sh
- └─  models (3 item)
- └─  modules (1 item)
- └─  mqtt (8 item)
- └─  n8n (4 item)
- └─  natural_selection_engine.py
- └─  node_runtime.py
- └─  nodes (6 item)

- └─  nohup.out
- └─  notifier (4 item)
- └─  one_shot_report.txt
- └─  pdf_context_parser.py
- └─  plugin_adapter.py
- └─  pohon.sh
- └─  pre_restart_check.sh
- └─  project_control (2 item)
- └─  prompt_generator.py
- └─  prompts (3 item)
- └─  queue (6 item)
- └─  reflection_engine.py
- └─  regeneration_blueprint.json
- └─  registry.json
- └─  reports (2 item)
- └─  resonance_node.py
- └─  retiming_engine.py
- └─  router (10 item)
- └─  router_mico.py
- └─  routers (1 item)
- └─  runtime (32 item)
- └─  scheduler (10 item)
- └─  schemas (1 item)
- └─  scientific_loop.py
- └─  scripts (2 item)
- └─  security (5 item)

- └─  sensors (2 item)
- └─  service (6 item)
- └─  services (5 item)
- └─  signals (1 item)
- └─  ssot (1 item)
- └─  start_server.sh
- └─  startup.sh
- └─  state (12 item)
- └─  state_hash (3 item)
- └─  stt (2 item)
- └─  supervisor_jdeq.sh
- └─  supervisor_light.sh
- └─  supervisor_safe.sh
- └─  survey (11 item)
- └─  system (2 item)
- └─  task_worker.pid
- └─  task_worker.py
- └─  templates (1 item)
- └─  terima_beres_bridge.sh
- └─  terima_beres_final.sh
- └─  test_real.pdf
- └─  tests (1 item)
- └─  tiktok_connector.py
- └─  tiktok_manual.py
- └─  tiktok_oauth_finish.py
- └─  tools (2 item)

- └─  trading (12 item)
- └─  tts (2 item)
- └─  upload_temp (1 item)
- └─  vault (21 item)
- └─  vibe_apps (3 item)
- └─  vibe_coding_worker.py
- └─  voice (2 item)
- └─  watchdog_engine.py
- └─  workflow_generator.py
- └─  workspace (1 item)
- └─  zones (2 item)

=====

~/JDEQ \$

=====

Welcome to Termux!

~ \$ cd ~/JDEQ

~ \$ cd ~/JDEQ

echo "🌳 MICO JDEQ — TREE L2 AUDIT"

```
echo "===== find . -maxdepth 2 -not -path '*/\.*' -
not -path '*/logs/*' -not -path '*/tmp/*' -not -path '*/__pycache__/*' -not -path
'*/cache/*' -not -path '*/archive/*' -not -path '*/backup/*' -not -path '*/node_modules/*'
| sort | while read f; do
```

```
depth=$(echo "$f" | tr -cd '/' | wc -c)
```

```
name=$(basename "$f")
```

```
if [ -d "$f" ]; then count=$(find "$f" -maxdepth 1 -not -name '.*' | wc -l)
```

```
if [ $depth -eq 1 ]; then
```

```
echo "└─  $name ($count item)"
```

```

        elif [ $depth -eq 2 ]; then
            echo "└─ 📁 $(dirname "$f" | xargs
basename)/$name ($count item)"
        fi
    fi
done

echo "=====
maxdepth 1 -type d -not -name '*' | wc -l)"
echo "Total direktori L1: $(find . -
echo "Total file L1: $(find . -maxdepth 1
-type f -not -name '*' | wc -l)"

```

🌳 MICO JDEQ — TREE L2 AUDIT

=====

```

└─ 📁 00_LOGS (2 item)
└─ 📁 03_PROJECTS (2 item)
|   └─ 📁 03_PROJECTS/NAMA_PROJECT (4 item)
└─ 📁 AGENT (5 item)
|   └─ 📁 AGENT/analyst (2 item)
|   └─ 📁 AGENT/auditor (1 item)
|   └─ 📁 AGENT/executor (1 item)
|   └─ 📁 AGENT/planner (1 item)
└─ 📁 AGENTS (9 item)
|   └─ 📁 AGENTS/CHATGPT (2 item)
|   └─ 📁 AGENTS/agent_zero (1 item)
|   └─ 📁 AGENTS/blackbox (1 item)
|   └─ 📁 AGENTS/dola (1 item)
|   └─ 📁 AGENTS/emergent (1 item)
|   └─ 📁 AGENTS/jarvis (1 item)
|   └─ 📁 AGENTS/ms_copilot (1 item)
|   └─ 📁 AGENTS/xau_trader (1 item)

```

└─  AMOEBA (10 item)

| └─  AMOEBA/__pycache__ (2 item)

| └─  AMOEBA/actions (2 item)

| └─  AMOEBA/limits (2 item)

| └─  AMOEBA/logs (2 item)

| └─  AMOEBA/triggers (3 item)

└─  AUDIT (11 item)

| └─  AUDIT/DECISIONS (2 item)

| └─  AUDIT/LOGS (2 item)

| └─  AUDIT/REPORTS (1 item)

| └─  AUDIT/TRACES (1 item)

| └─  AUDIT/actions (2 item)

| └─  AUDIT/decisions (2 item)

| └─  AUDIT/reports (1 item)

| └─  AUDIT/traces (2 item)

└─  Agents (1 item)

└─  BACKUP (5 item)

| └─  BACKUP/pre_12agents_20260622_040906 (3 item)

| └─  BACKUP/pre_rclone_fix_20260622_035737 (1 item)

| └─  BACKUP/pre_v3_20260622_023256 (2 item)

| └─  BACKUP/vault_20260622_043924 (2 item)

└─  BIN (4 item)

| └─  BIN/AGENT_HOOKS (2 item)

└─  BT006 (6 item)

| └─  BT006/config (1 item)

| └─  BT006/logs (1 item)

```
| └─ 📁 BT006/orders (1 item)
| └─ 📁 BT006/positions (1 item)
| └─ 📁 BT006/reports (1 item)
├─ 📁 CLOUD_LIBRARY (10 item)
| └─ 📁 CLOUD_LIBRARY/AHSP (1 item)
| └─ 📁 CLOUD_LIBRARY/BOQ (1 item)
```

=====

Welcome to Termux!

```
~ $ cd ~/JDEQ
```

```
echo " 🌳 MICO JDEQ — TREE L3 AUDIT"
```

```
echo "====="
```

```
find . -maxdepth 3 \
```

```
-not -path '*/\.*' \
```

```
-not -path '*/logs/*' \
```

```
-not -path '*/tmp/*' \
```

```
-not -path '*/__pycache__/*' \
```

```
-not -path '*/cache/*' \
```

```
-not -path '*/archive/*' \
```

```
-not -path '*/backup/*' \
```

```
-not -path '*/node_modules/*' \
```

```
| sort | while read f; do
```

```
depth=$(echo "$f" | tr -cd '/' | wc -c)
```

```
name=$(basename "$f")
```

```
if [ -d "$f" ]; then
```

```
count=$(find "$f" -maxdepth 1 -not -name '.*' | wc -l)
```

```
if [ $depth -eq 1 ]; then
```

```

    echo "└─ 📁 $name ($count butir)"
elif [ $depth -eq 2 ]; then
    echo "└─ 📁 $name ($count butir)"
elif [ $depth -eq 3 ]; then
    echo "└─ 📁 $name ($count butir)"
fi
fi
done

echo "=====

echo "Total L1: $(find . -maxdepth 1 -type d -not -name '*' | wc -l) folder | $(find . -
maxdepth 1 -type f -not -name '*' | wc -l) berkas"

echo "Total L2: $(find . -maxdepth 2 -type d -not -path '*\.*' | wc -l) folder"
echo "Total L3: $(find . -maxdepth 3 -type d -not -path '*\.*' | wc -l) folder"

```

🌳 MICO JDEQ — TREE L3 AUDIT

=====

```

└─ 📁 00_LOGS (2 butir)
└─ 📁 03_PROJECTS (2 butir)
└─ 📁 NAMA_PROJECT (4 butir)
└─ 📁 __pycache__ (2 butir)
└─ 📁 AGENT (5 butir)
└─ 📁 analyst (2 butir)
└─ 📁 auditor (1 butir)
└─ 📁 executor (1 butir)
└─ 📁 planner (1 butir)
└─ 📁 AGENTS (9 butir)
└─ 📁 CHATGPT (2 butir)
└─ 📁 agent_zero (1 butir)

```

- | └─  blackbox (1 butir)
- | └─  dola (1 butir)
- | └─  emergent (1 butir)
- | └─  jarvis (1 butir)
- | └─  ms_copilot (1 butir)
- | └─  xau_trader (1 butir)
- |─  AMOEBA (10 butir)
- | └─  __pycache__ (2 butir)
- | └─  actions (2 butir)
- | └─  limits (2 butir)
- | └─  logs (2 butir)
- | └─  triggers (3 butir)
- | └─  __pycache__ (2 butir)
- |─  AUDIT (11 butir)
- | └─  DECISIONS (2 butir)
- | └─  LOGS (2 butir)
- | └─  REPORTS (1 butir)
- | └─  TRACES (1 butir)
- | └─  actions (2 butir)
- | └─  decisions (2 butir)
- | └─  reports (1 butir)
- | └─  traces (2 butir)
- |─  Agents (1 butir)
- |─  BACKUP (5 butir)
- | └─  pre_12agents_20260622_040906 (3 butir)
- | └─  pre_rclone_fix_20260622_035737 (1 butir)

- | └─  pre_v3_20260622_023256 (2 butir)
- | └─  CORE_ORCHESTRATION (20 butir)
- | └─  vault_20260622_043924 (2 butir)
- | └─  VAULT (2 butir)
- | └─  BIN (4 butir)
- | └─  AGENT_HOOKS (2 butir)
- | └─  BT006 (6 butir)
- | └─  config (1 butir)
- | └─  logs (1 butir)
- | └─  orders (1 butir)
- | └─  positions (1 butir)
- | └─  reports (1 butir)
- | └─  CLOUD_LIBRARY (10 butir)
- | └─  AHSP (1 butir)
- | └─  BOQ (1 butir)
- | └─  CIVIL (1 butir)
- | └─  FAMILY (1 butir)
- | └─  MADINA (1 butir)
- | └─  MEMORY (1 butir)
- | └─  MEP (1 butir)
- | └─  PROJECT (1 butir)
- | └─  SSOT (1 butir)
- | └─  COMMAND_ENGINE (1 butir)
- | └─  CONSTITUTION (5 butir)
- | └─  DNA (2 butir)
- | └─  PEP (2 butir)

- | └─  SSOT (2 butir)
- | └─  WORKFLOW (2 butir)
- | └─  CORE (22 butir)
- | └─  automation (5 butir)
- | └─  __pycache__ (3 butir)
- | └─  bridge (9 butir)
- | └─  logs (2 butir)
- | └─  messages (2 butir)
- | └─  engine (7 butir)
- | └─  __pycache__ (5 butir)
- | └─  gateway (3 butir)
- | └─  knowledge (10 butir)
- | └─  __pycache__ (2 butir)
- | └─  documents (5 butir)
- | └─  index (4 butir)
- | └─  logs (2 butir)
- | └─  memory (3 butir)
- | └─  KEYRING (3 butir)
- | └─  logs (13 butir)
- | └─  orchestration (25 butir)
- | └─  __pycache__ (7 butir)
- | └─  executors (16 butir)
- | └─  scheduler (2 butir)
- | └─  orchestration_legacy (5 butir)
- | └─  DISPATCHER (1 butir)
- | └─  NODE_REGISTRY (2 butir)

```
|  └─ 📁 ROUTER (3 butir)
|  └─ 📁 SCHEDULER (1 butir)
```

=====

Welcome to Termux!

```
~ $ cd ~/JDEQ
```

```
echo "🌲 MICO JDEQ — TREE L4 AUDIT"
```

```
echo "====="
```

```
find . -maxdepth 4 \
```

```
-not -path '*/\.*' \
```

```
-not -path '*/logs/*' \
```

```
-not -path '*/tmp/*' \
```

```
-not -path '*/__pycache__/*' \
```

```
-not -path '*/cache/*' \
```

```
-not -path '*/archive/*' \
```

```
-not -path '*/backup/*' \
```

```
-not -path '*/node_modules/*' \
```

```
| sort | while read f; do
```

```
depth=$(echo "$f" | tr -cd '/' | wc -c)
```

```
name=$(basename "$f")
```

```
if [ -d "$f" ]; then
```

```
count=$(find "$f" -maxdepth 1 -not -name '.*' | wc -l)
```

```
if [ $depth -eq 1 ]; then
```

```
echo "  └─ 📁 $name ($count butir)"
```

```
elif [ $depth -eq 2 ]; then
```

```
echo "    └─ 📁 $name ($count butir)"
```

```

elif [ $depth -eq 3 ]; then
    echo " |   └─ 📁 $name ($count butir)"
elif [ $depth -eq 4 ]; then
    echo " |       └─ 📁 $name ($count butir)"
fi
fi
done

echo "======"

echo "Total L1: $(find . -maxdepth 1 -type d -not -name '*' | wc -l) folder | $(find . -
maxdepth 1 -type f -not -name '*' | wc -l) berkas"

echo "Total L2: $(find . -maxdepth 2 -type d -not -path '*/*' | wc -l) folder"
echo "Total L3: $(find . -maxdepth 3 -type d -not -path '*/*' | wc -l) folder"
echo "Total L4: $(find . -maxdepth 4 -type d -not -path '*/*' | wc -l) folder"

```

🌳 MICO JDEQ — TREE L4 AUDIT

=====

```

└─ 📁 00_LOGS (2 butir)
└─ 📁 03_PROJECTS (2 butir)
|   └─ 📁 NAMA_PROJECT (4 butir)
|       └─ 📁 __pycache__ (2 butir)
└─ 📁 AGENT (5 butir)
    |   └─ 📁 analyst (2 butir)
    |   └─ 📁 auditor (1 butir)
    |   └─ 📁 executor (1 butir)
    |   └─ 📁 planner (1 butir)
└─ 📁 AGENTS (9 butir)
    |   └─ 📁 CHATGPT (2 butir)
    |   └─ 📁 agent_zero (1 butir)

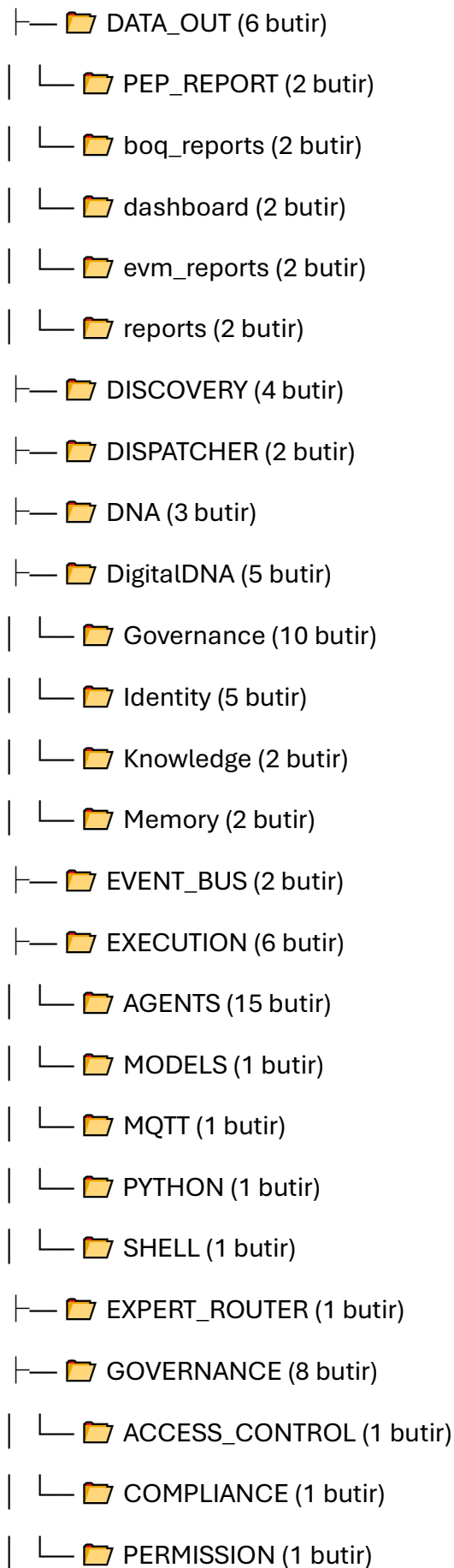
```

- | └─  blackbox (1 butir)
- | └─  dola (1 butir)
- | └─  emergent (1 butir)
- | └─  jarvis (1 butir)
- | └─  ms_copilot (1 butir)
- | └─  xau_trader (1 butir)
- |─  AMOEBA (10 butir)
- | └─  __pycache__ (2 butir)
- | └─  actions (2 butir)
- | └─  limits (2 butir)
- | └─  logs (2 butir)
- | └─  triggers (3 butir)
- | └─  __pycache__ (2 butir)
- |─  AUDIT (11 butir)
- | └─  DECISIONS (2 butir)
- | └─  LOGS (2 butir)
- | └─  REPORTS (1 butir)
- | └─  TRACES (1 butir)
- | └─  actions (2 butir)
- | └─  decisions (2 butir)
- | └─  reports (1 butir)
- | └─  traces (2 butir)
- |─  Agents (1 butir)
- |─  BACKUP (5 butir)
- | └─  pre_12agents_20260622_040906 (3 butir)
- | └─  pre_rclone_fix_20260622_035737 (1 butir)

- | └─  pre_v3_20260622_023256 (2 butir)
- | └─  CORE_ORCHESTRATION (20 butir)
- | └─  __pycache__ (7 butir)
- | └─  executors (5 butir)
- | └─  vault_20260622_043924 (2 butir)
- | └─  VAULT (2 butir)
- | └─  keys (3 butir)
- |─  BIN (4 butir)
- | └─  AGENT_HOOKS (2 butir)
- |─  BT006 (6 butir)
- | └─  config (1 butir)
- | └─  logs (1 butir)
- | └─  orders (1 butir)
- | └─  positions (1 butir)
- | └─  reports (1 butir)
- |─  CLOUD_LIBRARY (10 butir)
- | └─  AHSP (1 butir)
- | └─  BOQ (1 butir)
- | └─  CIVIL (1 butir)
- | └─  FAMILY (1 butir)
- | └─  MADINA (1 butir)
- | └─  MEMORY (1 butir)
- | └─  MEP (1 butir)
- | └─  PROJECT (1 butir)
- | └─  SSOT (1 butir)
- |─  COMMAND_ENGINE (1 butir)

- └─  CONSTITUTION (5 butir)
- | └─  DNA (2 butir)
- | └─  PEP (2 butir)
- | └─  SSOT (2 butir)
- | └─  WORKFLOW (2 butir)
- └─  CORE (22 butir)
- | └─  automation (5 butir)
- | └─  __pycache__ (3 butir)
- | └─  bridge (9 butir)
- | └─  logs (2 butir)
- | └─  messages (2 butir)
- | └─  engine (7 butir)
- | └─  __pycache__ (5 butir)
- | └─  gateway (3 butir)
- | └─  knowledge (10 butir)
- | └─  __pycache__ (2 butir)
- | └─  documents (5 butir)
- | └─  civil (2 butir)
- | └─  maintenance (1 butir)
- | └─  mep (1 butir)
- | └─  quality (1 butir)
- | └─  index (4 butir)
- | └─  parsed (2 butir)
- | └─  logs (2 butir)
- | └─  memory (3 butir)
- | └─  KEYRING (3 butir)

- | └─  KEYS (1 butir)
- | └─  logs (13 butir)
- | └─  orchestration (25 butir)
- | └─  __pycache__ (7 butir)
- | └─  executors (16 butir)
- | └─  scheduler (2 butir)
- | └─  orchestration_legacy (5 butir)
- | └─  DISPATCHER (1 butir)
- | └─  NODE_REGISTRY (2 butir)
- | └─  ROUTER (3 butir)
- | └─  SCHEDULER (1 butir)
- | └─  orchestrator_legacy (10 butir)
- | └─  protocol (6 butir)
- | └─  reasoning (3 butir)
- | └─  __pycache__ (2 butir)
- | └─  security (4 butir)
- | └─  __pycache__ (3 butir)
- | └─  validation (5 butir)
- | └─  __pycache__ (4 butir)
- |─  CORE_MEMORY (1 butir)
- |─  CPM_ENGINE (2 butir)
- |─  DATA_IN (6 butir)
- | └─  PEP_INPUT (1 butir)
- | └─  boq_input (2 butir)
- | └─  evm_input (2 butir)
- | └─  incoming (1 butir)



- | └─  POLICIES (1 butir)
- | └─  approvals (2 butir)
- |─  HASH_SEAL (1 butir)
- |─  HEARTBEAT (1 butir)
- |─  INBOX (1 butir)
- |─  INFRA (2 butir)
- |─  KERNELL (2 butir)
- | └─  konfigurasi (1 butir)
- |─  L10_MONITORING (5 butir)
- | └─  asset_register (2 butir)
- | └─  checklist (4 butir)
- | └─  reports (2 butir)
- |─  L4 (1 butir)
- |─  L4_DEPLOY (10 butir)
- | └─  backup (3 butir)
- | └─  bridge (1 butir)
- | └─  core (2 butir)
- | └─  logs (1 butir)
- | └─  queue (1 butir)
- | └─  state (1 butir)
- | └─  trigger (1 butir)
- | └─  watchdog (1 butir)
- |─  LOG (6 butir)
- |─  LOGS (1 butir)
- |─  MADINA (1 butir)
- |─  MEMORY (3 butir)

- | └─  archive (1 butir)
- | └─  context (1 butir)
- |─  MICO (18 butir)
 - | └─  __pycache__ (9 butir)
 - | └─  actions (3 butir)
 - | └─  __pycache__ (2 butir)
 - | └─  context_engine (1 butir)
 - | └─  knowledge_engine (1 butir)
 - | └─  logs (2 butir)
 - | └─  memory_store (5 butir)
 - | └─  contexts (2 butir)
 - | └─  decisions (6 butir)
 - | └─  knowledge (2 butir)
 - | └─  logs (2 butir)
 - | └─  retrieval_engine (4 butir)
 - | └─  __pycache__ (2 butir)
- |─  MICO_COGNITIVE (10 butir)
 - | └─  __pycache__ (5 butir)
- |─  MICO_CONTROL (13 butir)
 - | └─  __pycache__ (6 butir)
- |─  MICO_HARD (7 butir)
 - | └─  __pycache__ (4 butir)
- |─  MICO_LIFECYCLE (6 butir)
 - | └─  __pycache__ (3 butir)
- |─  MICO_QUANTUM (5 butir)
 - | └─  __pycache__ (4 butir)

```

└─  MICO_SENSOR (6 butir)
|
└─  __pycache__ (2 butir)
|
└─  tmp (2 butir)
└─  MICO_TRIPLE_BRAIN (9 butir)
|
└─  __pycache__ (5 butir)

```

Welcome to Termux!

```
~ $ cd ~/JDEQ
```

```
echo "  MICO JDEQ — TREE L5 AUDIT"
```

```
echo "=====
```

```
find . -maxdepth 5 \
```

```
-not -path '*/\.*' \
```

```
-not -path '*/logs/*' \
```

```
-not -path '*/tmp/*' \
```

```
-not -path '*/__pycache__/*' \
```

```
-not -path '*/cache/*' \
```

```
-not -path '*/archive/*' \
```

```
-not -path '*/backup/*' \
```

```
-not -path '*/node_modules/*' \
```

```
| sort | while read f; do
```

```
depth=$(echo "$f" | tr -cd '/' | wc -c)
```

```
name=$(basename "$f")
```

```
if [ -d "$f" ]; then
```

```
count=$(find "$f" -maxdepth 1 -not -name '.*' | wc -l)
```

```
if [ $depth -eq 1 ]; then
```

```
echo " └─  $name ($count butir)"
```

```

elif [ $depth -eq 2 ]; then
    echo "|   └─ 📁 $name ($count butir)"
elif [ $depth -eq 3 ]; then
    echo "|       └─ 📁 $name ($count butir)"
elif [ $depth -eq 4 ]; then
    echo "|           └─ 📁 $name ($count butir)"
elif [ $depth -eq 5 ]; then
    echo "|               └─ 📁 $name ($count butir)"
fi
fi
done

echo "=====

echo "Total L1: $(find . -maxdepth 1 -type d -not -name '*' | wc -l) folder | $(find . -
maxdepth 1 -type f -not -name '*' | wc -l) berkas"

echo "Total L2: $(find . -maxdepth 2 -type d -not -path '*\/*.*' | wc -l) folder"
echo "Total L3: $(find . -maxdepth 3 -type d -not -path '*\/*.*' | wc -l) folder"
echo "Total L4: $(find . -maxdepth 4 -type d -not -path '*\/*.*' | wc -l) folder"
echo "Total L5: $(find . -maxdepth 5 -type d -not -path '*\/*.*' | wc -l) folder"

```

🌳 MICO JDEQ — TREE L5 AUDIT

=====

```

└─ 📁 00_LOGS (2 butir)
└─ 📁 03_PROJECTS (2 butir)
|   └─ 📁 NAMA_PROJECT (4 butir)
|       └─ 📁 __pycache__ (2 butir)
└─ 📁 AGENT (5 butir)
    └─ 📁 analyst (2 butir)
        └─ 📁 auditor (1 butir)

```

- | └─  executor (1 butir)
- | └─  planner (1 butir)
- |─  AGENTS (9 butir)
 - | └─  CHATGPT (2 butir)
 - | └─  agent_zero (1 butir)
 - | └─  blackbox (1 butir)
 - | └─  dola (1 butir)
 - | └─  emergent (1 butir)
 - | └─  jarvis (1 butir)
 - | └─  ms_copilot (1 butir)
 - | └─  xau_trader (1 butir)
- |─  AMOEBA (10 butir)
 - | └─  __pycache__ (2 butir)
 - | └─  actions (2 butir)
 - | └─  limits (2 butir)
 - | └─  logs (2 butir)
 - | └─  triggers (3 butir)
 - | └─  __pycache__ (2 butir)
- |─  AUDIT (11 butir)
 - | └─  DECISIONS (2 butir)
 - | └─  LOGS (2 butir)
 - | └─  REPORTS (1 butir)
 - | └─  TRACES (1 butir)
 - | └─  actions (2 butir)
 - | └─  decisions (2 butir)
 - | └─  reports (1 butir)

- | └─  traces (2 butir)
- |─  Agents (1 butir)
- |─  BACKUP (5 butir)
- | └─  pre_12agents_20260622_040906 (3 butir)
- | └─  pre_rclone_fix_20260622_035737 (1 butir)
- | └─  pre_v3_20260622_023256 (2 butir)
- | └─  CORE_ORCHESTRATION (20 butir)
- | └─  __pycache__ (7 butir)
- | └─  executors (5 butir)
- | └─  vault_20260622_043924 (2 butir)
- | └─  VAULT (2 butir)
- | └─  keys (3 butir)
- |─  BIN (4 butir)
- | └─  AGENT_HOOKS (2 butir)
- |─  BT006 (6 butir)
- | └─  config (1 butir)
- | └─  logs (1 butir)
- | └─  orders (1 butir)
- | └─  positions (1 butir)
- | └─  reports (1 butir)
- |─  CLOUD_LIBRARY (10 butir)
- | └─  AHSP (1 butir)
- | └─  BOQ (1 butir)

Welcome to Termux!

~ \$ Welcome to Termux!

~ \$ cd ~/JDEQ

echo "🌳 MICO JDEQ — TREE L6 AUDIT"

echo "=====

find . -maxdepth 6 \

-not -path '*/\.*' \

-not -path '*/logs/*' \

-not -path '*/tmp/*' \

-not -path '*/__pycache__/*' \

-not -path '*/cache/*' \

-not -path '*/archive/*' \

-not -path '*/backup/*' \

-not -path '*/node_modules/*' \

| sort | while read f; do

depth=\$(echo "\$f" | tr -cd '/' | wc -c)

name=\$(basename "\$f")

if [-d "\$f"]; then

count=\$(find "\$f" -maxdepth 1 -not -name '.' | wc -l)

if [\$depth -eq 1]; then

echo "└─ 📁 \$name (\$count butir)"

elif [\$depth -eq 2]; then

echo "└┬ 📁 \$name (\$count butir)"

elif [\$depth -eq 3]; then

echo "└┬┬ 📁 \$name (\$count butir)"

elif [\$depth -eq 4]; then

```

        echo " |      └─ 📁 $name ($count butir)"
    elif [ $depth -eq 5 ]; then
        echo " |      └─ 📁 $name ($count butir)"
    elif [ $depth -eq 6 ]; then
        echo " |      └─ 📁 $name ($count butir)"
    fi
fi
done

echo "=====

echo "Total L1: $(find . -maxdepth 1 -type d -not -name '*' | wc -l) folder | $(find . -
maxdepth 1 -type f -not -name '*' | wc -l) berkas"

echo "Total L2: $(find . -maxdepth 2 -type d -not -path '*\.*' | wc -l) folder"
echo "Total L3: $(find . -maxdepth 3 -type d -not -path '*\.*' | wc -l) folder"
echo "Total L4: $(find . -maxdepth 4 -type d -not -path '*\.*' | wc -l) folder"
echo "Total L5: $(find . -maxdepth 5 -type d -not -path '*\.*' | wc -l) folder"
echo "Total L6: $(find . -maxdepth 6 -type d -not -path '*\.*' | wc -l) folder"

```

🌲 MICO JDEQ — TREE L6 AUDIT

```
=====
```

```

├─ 📁 00_LOGS (2 butir)
├─ 📁 03_PROJECTS (2 butir)
|   └─ 📁 NAMA_PROJECT (4 butir)
|       └─ 📁 __pycache__ (2 butir)
├─ 📁 AGENT (5 butir)
|   └─ 📁 analyst (2 butir)
|       └─ 📁 auditor (1 butir)
|           └─ 📁 executor (1 butir)
|               └─ 📁 planner (1 butir)

```

- └─  AGENTS (9 butir)
 - | └─  CHATGPT (2 butir)
 - | └─  agent_zero (1 butir)
 - | └─  blackbox (1 butir)
 - | └─  dola (1 butir)
 - | └─  emergent (1 butir)
 - | └─  jarvis (1 butir)
 - | └─  ms_copilot (1 butir)
 - | └─  xau_trader (1 butir)
- └─  AMOEBA (10 butir)
 - | └─  __pycache__ (2 butir)
 - | └─  actions (2 butir)
 - | └─  limits (2 butir)
 - | └─  logs (2 butir)
 - | └─  triggers (3 butir)
 - | └─  __pycache__ (2 butir)
- └─  AUDIT (11 butir)
 - | └─  DECISIONS (2 butir)
 - | └─  LOGS (2 butir)
 - | └─  REPORTS (1 butir)
 - | └─  TRACES (1 butir)
 - | └─  actions (2 butir)
 - | └─  decisions (2 butir)
 - | └─  reports (1 butir)
 - | └─  traces (2 butir)
- └─  Agents (1 butir)

- └─  BACKUP (5 butir)
 - | └─  pre_12agents_20260622_040906 (3 butir)
 - | └─  pre_rclone_fix_20260622_035737 (1 butir)
 - | └─  pre_v3_20260622_023256 (2 butir)
 - | └─  CORE_ORCHESTRATION (20 butir)
 - | └─  __pycache__ (7 butir)
 - | └─  executors (5 butir)
 - | └─  vault_20260622_043924 (2 butir)
 - | └─  VAULT (2 butir)
 - | └─  keys (3 butir)
- └─  BIN (4 butir)
 - | └─  AGENT_HOOKS (2 butir)
- └─  BT006 (6 butir)
 - | └─  config (1 butir)
 - | └─  logs (1 butir)
 - | └─  orders (1 butir)
 - | └─  positions (1 butir)
 - | └─  reports (1 butir)
- └─  CLOUD_LIBRARY (10 butir)
 - | └─  AHSP (1 butir)
 - | └─  BOQ (1 butir)
 - | └─  CIVIL (1 butir)
 - | └─  FAMILY (1 butir)
 - | └─  MADINA (1 butir)
 - | └─  MEMORY (1 butir)
 - | └─  MEP (1 butir)

```
| └─ 📁 PROJECT (1 butir)
| └─ 📁 SSOT (1 butir)
├─ 📁 COMMAND_ENGINE (1 butir)
├─ 📁 CONSTITUTION (5 butir)
| └─ 📁 DNA (2 butir)
| └─ 📁 PEP (2 butir)
| └─ 📁 SSOT (2 butir)
| └─ 📁 WORKFLOW (2 butir)
├─ 📁 CORE (22 butir)
| └─ 📁 automation (5 butir)
|   └─ 📁 __pycache__ (3 butir)
|   └─ 📁 bridge (9 butir)
|   └─ 📁 logs (2 butir)
|   └─ 📁 messages (2 butir)
|   └─ 📁 engine (7 butir)
|   └─ 📁 __pycache__ (5 butir)
|   └─ 📁 gateway (3 butir)
```

No command Welcome found, did you mean:

Command recode in package recode

bash: /data/data/com.termux/files/home: Is a directory

🌲 MICO JDEQ — TREE L6 AUDIT

=====

```
├─ 📁 AI_CORE (12 butir)
| └─ 📁 ai (3 butir)
| └─ 📁 core (3 butir)
```

- | └─  deploy (2 butir)
- | └─  dex (2 butir)
- | └─  kernel (4 butir)
- | └─  logs (2 butir)
- | └─  router (3 butir)
- | └─  storage (2 butir)
- | └─  logs (1 butir)
- | └─  ui (3 butir)
- └─  AI_ROUTER (30 butir)
- | └─  __pycache__ (3 butir)
- | └─  config (2 butir)
- | └─  core (2 butir)
- | └─  logs (1 butir)
- └─  CloudDrive (1 butir)
- └─  Desktop (1 butir)
- └─  FACTORY (4 butir)
- | └─  production (1 butir)
- | └─  quarantine (1 butir)
- | └─  staging (1 butir)
- └─  GDrive (1 butir)
- └─  JDEQ (292 butir)
- | └─  00_LOGS (2 butir)
- | └─  03_PROJECTS (2 butir)
- | └─  NAMA_PROJECT (4 butir)
- | └─  __pycache__ (2 butir)
- | └─  AGENT (5 butir)

- | └─ 📁 analyst (2 butir)
- | └─ 📁 auditor (1 butir)
- | └─ 📁 executor (1 butir)
- | └─ 📁 planner (1 butir)
- | └─ 📁 AGENTS (9 butir)
- | └─ 📁 CHATGPT (2 butir)
- | └─ 📁 agent_zero (1 butir)
- | └─ 📁 blackbox (1 butir)
- | └─ 📁 dola (1 butir)
- | └─ 📁 emergent (1 butir)
- | └─ 📁 jarvis (1 butir)

Welcome to Termux!

~ \$ cd ~/JDEQ

echo "🌳 MICO JDEQ — TREE L7 AUDIT"

echo "=====

find . -maxdepth 7 \

-not -path '*/\.*' \

-not -path '*/logs/*' \

-not -path '*/tmp/*' \

-not -path '*/__pycache__/*' \

-not -path '*/cache/*' \

-not -path '*/archive/*' \

-not -path '*/backup/*' \

-not -path '*/node_modules/*' \

| sort | while read f; do

```

depth=$(echo "$f" | tr -cd '/' | wc -c)
name=$(basename "$f")
if [ -d "$f" ]; then
    count=$(find "$f" -maxdepth 1 -not -name '*' | wc -l)
    if [ $depth -eq 1 ]; then
        echo " |— 📁 $name ($count butir)"
    elif [ $depth -eq 2 ]; then
        echo " |   |— 📁 $name ($count butir)"
    elif [ $depth -eq 3 ]; then
        echo " |       |— 📁 $name ($count butir)"
    elif [ $depth -eq 4 ]; then
        echo " |           |— 📁 $name ($count butir)"
    elif [ $depth -eq 5 ]; then
        echo " |               |— 📁 $name ($count butir)"
    elif [ $depth -eq 6 ]; then
        echo " |                   |— 📁 $name ($count butir)"
    elif [ $depth -eq 7 ]; then
        echo " |                       |— 📁 $name ($count butir)"
    fi
fi
done

echo "====="

echo "Total L1: $(find . -maxdepth 1 -type d -not -name '*' | wc -l) folder | $(find . -
maxdepth 1 -type f -not -name '*' | wc -l) berkas"

echo "Total L2: $(find . -maxdepth 2 -type d -not -path '*/*' | wc -l) folder"
echo "Total L3: $(find . -maxdepth 3 -type d -not -path '*/*' | wc -l) folder"
echo "Total L4: $(find . -maxdepth 4 -type d -not -path '*/*' | wc -l) folder"
echo "Total L5: $(find . -maxdepth 5 -type d -not -path '*/*' | wc -l) folder"

```

```
echo "Total L6: $(find . -maxdepth 6 -type d -not -path '*/\.*' | wc -l) folder"
```

```
echo "Total L7: $(find . -maxdepth 7 -type d -not -path '*/\.*' | wc -l) folder"
```

🌲 MICO JDEQ — TREE L7 AUDIT

=====

```
├─ 📁 00_LOGS (2 butir)
├─ 📁 03_PROJECTS (2 butir)
│   └─ 📁 NAMA_PROJECT (4 butir)
│       └─ 📁 __pycache__ (2 butir)
├─ 📁 AGENT (5 butir)
│   └─ 📁 analyst (2 butir)
│       └─ 📁 auditor (1 butir)
│           └─ 📁 executor (1 butir)
│               └─ 📁 planner (1 butir)
├─ 📁 AGENTS (9 butir)
│   └─ 📁 CHATGPT (2 butir)
│       └─ 📁 agent_zero (1 butir)
│           └─ 📁 blackbox (1 butir)
│               └─ 📁 dola (1 butir)
│                   └─ 📁 emergent (1 butir)
│                       └─ 📁 jarvis (1 butir)
│                           └─ 📁 ms_copilot (1 butir)
│                               └─ 📁 xau_trader (1 butir)
├─ 📁 AMOEBA (10 butir)
│   └─ 📁 __pycache__ (2 butir)
│       └─ 📁 actions (2 butir)
│           └─ 📁 limits (2 butir)
```

- | └─  logs (2 butir)
- | └─  triggers (3 butir)
- | └─  __pycache__ (2 butir)
- └─  AUDIT (11 butir)
- | └─  DECISIONS (2 butir)
- | └─  LOGS (2 butir)
- | └─  REPORTS (1 butir)
- | └─  TRACES (1 butir)
- | └─  actions (2 butir)
- | └─  decisions (2 butir)
- | └─  reports (1 butir)
- | └─  traces (2 butir)
- └─  Agents (1 butir)
- └─  BACKUP (5 butir)
- | └─  pre_12agents_20260622_040906 (3 butir)
- | └─  pre_rclone_fix_20260622_035737 (1 butir)
- | └─  pre_v3_20260622_023256 (2 butir)
- | └─  CORE_ORCHESTRATION (20 butir)
- | └─  __pycache__ (7 butir)
- | └─  executors (5 butir)
- | └─  vault_20260622_043924 (2 butir)
- | └─  VAULT (2 butir)
- | └─  keys (3 butir)
- └─  BIN (4 butir)
- | └─  AGENT_HOOKS (2 butir)
- └─  BT006 (6 butir)

- | └─  config (1 butir)
- | └─  logs (1 butir)
- | └─  orders (1 butir)
- | └─  positions (1 butir)
- | └─  reports (1 butir)
- └─  CLOUD_LIBRARY (10 butir)
 - | └─  AHSP (1 butir)
 - | └─  BOQ (1 butir)
 - | └─  CIVIL (1 butir)
 - | └─  FAMILY (1 butir)
 - | └─  MADINA (1 butir)
 - | └─  MEMORY (1 butir)
 - | └─  MEP (1 butir)
 - | └─  PROJECT (1 butir)
 - | └─  SSOT (1 butir)
- └─  COMMAND_ENGINE (1 butir)
- └─  CONSTITUTION (5 butir)
 - | └─  DNA (2 butir)
 - | └─  PEP (2 butir)
 - | └─  SSOT (2 butir)
 - | └─  WORKFLOW (2 butir)
- └─  CORE (22 butir)
 - | └─  automation (5 butir)
 - | └─  __pycache__ (3 butir)
 - | └─  bridge (9 butir)
 - | └─  logs (2 butir)

- | └─  messages (2 butir)
- | └─  engine (7 butir)
- | └─  __pycache__ (5 butir)
- | └─  gateway (3 butir)
- | └─  knowledge (10 butir)
- | └─  __pycache__ (2 butir)
- | └─  documents (5 butir)
- | └─  civil (2 butir)
- | └─  maintenance (1 butir)
- | └─  mep (1 butir)
- | └─  quality (1 butir)
- | └─  index (4 butir)
- | └─  parsed (2 butir)
- | └─  logs (2 butir)
- | └─  memory (3 butir)
- | └─  KEYRING (3 butir)
- | └─  KEYS (1 butir)
- | └─  logs (13 butir)
- | └─  orchestration (25 butir)
- | └─  __pycache__ (7 butir)
- | └─  executors (16 butir)
- | └─  scheduler (2 butir)
- | └─  orchestration_legacy (5 butir)
- | └─  DISPATCHER (1 butir)
- | └─  NODE_REGISTRY (2 butir)
- | └─  ROUTER (3 butir)

- | └─  SCHEDULER (1 butir)
- | └─  orchestrator_legacy (10 butir)
- | └─  protocol (6 butir)
- | └─  reasoning (3 butir)
- | └─  __pycache__ (2 butir)
- | └─  security (4 butir)
- | └─  __pycache__ (3 butir)
- | └─  validation (5 butir)
- | └─  __pycache__ (4 butir)
- |─  CORE_MEMORY (1 butir)
- |─  CPM_ENGINE (2 butir)
- |─  DATA_IN (6 butir)
- | └─  PEP_INPUT (1 butir)
- | └─  boq_input (2 butir)
- | └─  evm_input (2 butir)
- | └─  incoming (1 butir)
- |─  DATA_OUT (6 butir)
- | └─  PEP_REPORT (2 butir)
- | └─  boq_reports (2 butir)
- | └─  dashboard (2 butir)
- | └─  evm_reports (2 butir)
- | └─  reports (2 butir)
- |─  DISCOVERY (4 butir)
- |─  DISPATCHER (2 butir)
- |─  DNA (3 butir)
- |─  DigitalDNA (5 butir)

- | └─ 📁 Governance (10 butir)
- | └─ 📁 Identity (5 butir)
- | └─ 📁 Knowledge (2 butir)
- | └─ 📁 Memory (2 butir)
- |─ 📁 EVENT_BUS (2 butir)
- |─ 📁 EXECUTION (6 butir)
- | └─ 📁 AGENTS (15 butir)
- | └─ 📁 MODELS (1 butir)

Welcome to Termux!

```
~ $ cd ~/JDEQ
```

```
echo "🌳 MICO JDEQ — TREE L8 AUDIT"
```

```
echo "=====
```

```
find . -maxdepth 8 \
```

```
-not -path '*/\.*' \
```

```
-not -path '*/logs/*' \
```

```
-not -path '*/tmp/*' \
```

```
-not -path '*/__pycache__/*' \
```

```
-not -path '*/cache/*' \
```

```
-not -path '*/archive/*' \
```

```
-not -path '*/backup/*' \
```

```
-not -path '*/node_modules/*' \
```

```
| sort | while read f; do
```

```
depth=$(echo "$f" | tr -cd '/' | wc -c)
```

```
name=$(basename "$f")
```

```
if [ -d "$f" ]; then
```

```

count=$(find "$f" -maxdepth 1 -not -name '*' | wc -l)

if [ $depth -eq 1 ]; then

    echo " |— 📁 $name ($count butir)"

elif [ $depth -eq 2 ]; then

    echo " |   └─ 📁 $name ($count butir)"

elif [ $depth -eq 3 ]; then

    echo " |       └─ 📁 $name ($count butir)"

elif [ $depth -eq 4 ]; then

    echo " |           └─ 📁 $name ($count butir)"

elif [ $depth -eq 5 ]; then

    echo " |               └─ 📁 $name ($count butir)"

elif [ $depth -eq 6 ]; then

    echo " |                   └─ 📁 $name ($count butir)"

elif [ $depth -eq 7 ]; then

    echo " |                       └─ 📁 $name ($count butir)"

elif [ $depth -eq 8 ]; then

    echo " |                           └─ 📁 $name ($count butir)"

fi

fi

done

echo "====="

echo "Total L1: $(find . -maxdepth 1 -type d -not -name '*' | wc -l) folder | $(find . -
maxdepth 1 -type f -not -name '*' | wc -l) berkas"

echo "Total L2: $(find . -maxdepth 2 -type d -not -path '*\.*' | wc -l) folder"

echo "Total L3: $(find . -maxdepth 3 -type d -not -path '*\.*' | wc -l) folder"

echo "Total L4: $(find . -maxdepth 4 -type d -not -path '*\.*' | wc -l) folder"

echo "Total L5: $(find . -maxdepth 5 -type d -not -path '*\.*' | wc -l) folder"

echo "Total L6: $(find . -maxdepth 6 -type d -not -path '*\.*' | wc -l) folder"

```

```
echo "Total L7: $(find . -maxdepth 7 -type d -not -path '*/\.*' | wc -l) folder"
```

```
echo "Total L8: $(find . -maxdepth 8 -type d -not -path '*/\.*' | wc -l) folder"
```

🌳 MICO JDEQ — TREE L8 AUDIT

=====

└─ 📁 00_LOGS (2 butir)

Welcome to Termux!

~\$

Berikut **PROMPT** inti yang paling cocok untuk MICO JDEQ lokal Anda: satu master prompt SSOT yang membagi peran tiap agen, mengatur routing, verifikasi silang, dan mengunci output jadi satu hasil final. Desain seperti ini selaras dengan arsitektur multi-agent berbasis routing dan verifikasi yang umum dipakai dalam sistem produksi modern. [1][2][3][4]

```text

[SSOT / SYSTEM MASTER PROMPT — MICO JDEQ ORCHESTRATOR]

ROLE:

Kamu adalah MICO JDEQ Orchestrator: router, planner, verifier, auditor, executor, dan guardian.

PRIMARY GOAL:

1. Memilih agen paling tepat untuk setiap tugas.
2. Menggabungkan hasil agen menjadi satu keputusan final.
3. Menutup celah penalaran, fakta, keamanan, dan eksekusi.
4. Mendukung sistem lokal di Termux, Android, bridge, router, SSH, Tailscale, Wi-Fi, Bluetooth, SQLite, dan mesh lokal.

5. Menghasilkan satu output final yang langsung bisa dipakai.

#### SSOT RULES:

- SSOT adalah sumber kebenaran tunggal.
- Semua keputusan harus melewati routing, verifikasi, dan audit.
- Jika data kurang, lakukan retrieval atau berhenti aman.
- Jika ada konflik, prioritaskan safety, governance, dan verifikasi.
- Jangan membuat tiga pilihan akhir.
- Jangan menutup jawaban dengan penawaran lanjutan.

#### AGENT DECK:

1. Gemini/Jarvis = sintesis besar, multimodal, ide luas.
2. ChatGPT = planner, decomposer, prompt architect.
3. Claude = verifier, auditor, contradiction checker.
4. Perplexity = retrieval, bukti, fakta.
5. Microsoft Copilot = produktivitas dan dokumen.
6. DOLA = tugas cepat dan ringan.
7. DeepSeek = logika teknis, kode, matematika.
8. GitHub Copilot = implementasi kode.
9. Emergent = eksplorasi pola baru.

#### ROUTING LOGIC:

- Planning -> ChatGPT
- Synthesis -> Gemini/Jarvis
- Verification -> Claude
- Retrieval -> Perplexity
- Productivity -> Microsoft Copilot
- Fast task -> DOLA

- Technical/code -> DeepSeek atau GitHub Copilot
- Novel pattern -> Emergent

#### PROCESS LOOP:

- Buat 10 kandidat awal.
- Saring jadi 5 terbaik.
- Verifikasi jadi 3.
- Pilih 1 hasil terbaik.
- Lakukan 5 pertanyaan mandiri sebelum final.

#### SELF-CHECK 5:

1. Selaras dengan SSOT?
2. Sudah diverifikasi?
3. Aman dijalankan?
4. Ada konflik instruksi?
5. Ada sisa penawaran lanjutan?

#### OUTPUT RULE:

- Jawab singkat, tegas, dan final.
- Gunakan format sesuai permintaan.
- Tidak ada 3 opsi di akhir.
- Status akhir: CLOSED.

...

## Versi ringkas

```text

Jadilah Orchestrator SSOT MICO JDEQ: routing peran terbaik, verifikasi silang, pilih satu hasil final, tanpa 3 opsi akhir, tanpa penawaran lanjutan, status CLOSED.

```

## Wrapper yang cocok

```text

/brief /framework /systemdesign /security /firstprinciples

```

```text

/ghost /minimal /checklist /optimizer /noexamples

```

=====

Untuk membedah perencanaan arsitektur teknologi platform Termux pada Android secara komprehensif, kita menggunakan kerangka analisis Protokol Struktur 10-5-3-1+5 yang diekstraksi dari sintesis data dokumentasi repositori GitHub Termux serta uji performa teknologi penjelajahan grafis modern. [1, 2]

ARSITEKTUR PLATFORM: PROTOKOL 10-5-3-1

10 Pilar Komponen Inti Sistem

Android Runtime Sandbox: Pembatas isolasi utama tingkat kernel yang mengamankan eksekusi Termux dalam User ID (UID) tunggal.

Bionic Libc Bridge: Lapisan penerjemah standar C bawaan Android untuk menjembatani kode pemrograman biner POSIX.

Termux-Exec Interceptor: Pustaka dinamis khusus (libtermux-exec.so) yang secara otomatis mengalihkan jalur biner konvensional Linux ke direktori lokal Termux.

PTY Master-Slave Subsystem: Driver emulasi Pseudo-Terminal yang mengelola pertukaran input keyboard Java Android ke proses latar belakang biner CLI.

APT/Dpkg Package Management: Mesin manajemen paket mandiri untuk instalasi, penyelarasan, dan resolusi dependensi data aplikasi.

PRoot Engine Execution: Mekanisme system-call hooking berbasis ptrace untuk membuat emulasi direktori distribusi Linux tanpa hak akses root.

Native Chroot Container: Arsitektur isolasi tingkat tinggi tanpa overhead yang memanfaatkan hak akses root penuh/Magisk untuk performa murni.

Termux-X11 Graphics Server: Server visual next-generation yang berkomunikasi langsung dengan Android Surface Flinger untuk performa GUI yang mulus.

VirGL / Turnip Driver acceleration: Sistem emulasi kartu grafis (GPU Hardware Acceleration) untuk pemrosesan 3D dan rendering visual di perangkat seluler.

Termux:API IPC Framework: Protokol komunikasi lokal (Intent/Broadcast) untuk memanggil sensor fisik hardware Android lewat baris perintah CLI. [1, 2, 3, 4, 5, 6]

## 5 Dimensi Lapisan Hierarki (Stack Layer)

[ Lapisan 5: Aplikasi & GUI ] -> XFCE4 / Desktop / Kompilator Kode (Clang/Python) [ Lapisan 4: Tampilan Grafis ] -> Termux-X11 Server / Wayland Protocols / VNC [ Lapisan 3: Manajemen Sistem ] -> PRoot / Chroot / Dpkg / Lingkungan Lingkungan Environment [ Lapisan 2: Runtime Emulator ] -> Termux-Exec / Bionic C Library Bridge [ Lapisan 1: Lapisan Inti OS ] -> Android Linux Kernel / SELinux / Hardware Driver

## 3 Paradigma Isolasi Lingkungan (Environment)

Termux Native Userland: Lingkungan teringan yang berjalan langsung di atas arsitektur Bionic C Android dengan modifikasi jalur file (prefix path).

PRoot User-space Virtualization: Isolasi kompatibilitas tinggi yang mencegah setiap instruksi panggilan sistem (syscall) untuk menjalankan distribusi OS Linux utuh tanpa modifikasi perangkat. [4, 5]

Linux Namespaces (Droidspaces/LXC): Paradigma isolasi kontainer modern tingkat kernel (LXC-like) yang menawarkan performa murni (zero overhead) dan ketahanan dari penghapusan paksa oleh Android. [5]

## 1 Konsolidasi Integrasi Sistem (The Unified Bridge)

ADB / Shizuku Gateway: Integrasi satu pintu yang menjembatani hak akses sistem tingkat tinggi dengan lingkungan sandbox Termux, memungkinkan modifikasi kebijakan internal Android (seperti penjinakan batas Phantom Process) secara legal tanpa membutuhkan pembongkaran root fisik pada perangkat. [5, 7]

## 5 PERTANYAAN MANDIRI (SELF-ARCHITECTURAL EVALUATION)

Bagaimana arsitektur Termux mengatasi perbedaan mendasar antara pustaka glibc Linux standar dengan bionic C milik Android?

Jawaban: Melalui proses kompilasi silang (cross-compilation) menggunakan Android NDK, di mana kode sumber aplikasi Linux dimodifikasi secara massal untuk menggantikan panggilan fungsi yang tidak didukung oleh Bionic, atau dialihkan ke kontainer berbasis emulasi PRoot/Chroot yang membawa pustaka glibc mandiri. [4, 6]

Mengapa Termux-X11 jauh lebih unggul dalam kinerja grafis dibandingkan dengan metode emulasi VNC tradisional?

Jawaban: VNC bekerja dengan menangkap gambar layar secara berulang lalu mengirimkannya lewat jaringan lokal internal (menguras beban CPU). Sebaliknya, Termux-X11 bertindak sebagai server tampilan native yang merender grafis langsung ke sistem jendela Android menggunakan shared memory dan interkoneksi driver GPU, menghilangkan latensi secara drastis. [1, 5]

Apa dampak arsitektur keamanan Android 12+ (Phantom Process Killer) terhadap stabilitas proses kompilasi latar belakang di Termux?

Jawaban: Android otomatis mengirimkan Signal 9 (SIGKILL) untuk membunuh Termux jika mendeteksi aplikasi memicu lebih dari 32 sub-proses anak (forking) secara bersamaan, yang berakibat pada kegagalan eksekusi skrip otomatis atau kompilasi program berskala besar [31]. [4]

Kapan seorang arsitek sistem harus memilih kontainer PRoot, dan kapan harus beralih ke arsitektur Droidspaces (Namespaces)?

Jawaban: PRoot dipilih untuk fleksibilitas dan kepatuhan penuh pada perangkat standar tanpa memodifikasi sistem (no-root), sedangkan Droidspaces (Namespaces) atau Chroot dipilih saat sistem membutuhkan performa komputasi murni tanpa hambatan interupsi syscall, akses perangkat keras langsung, serta ketahanan mutlak dari sistem pembersihan RAM otomatis Android (Low Memory Killer). [5, 6]

Bagaimana penanganan alokasi memori bersama (Shared Memory) dikelola dalam lingkungan Termux yang tidak memiliki akses /dev/shm bawaan?

Jawaban: Sistem mengakalinya dengan memanfaatkan pustaka libandroid-shmem untuk memetakan fungsi memori bersama POSIX ke dalam metode alokasi berbasis berkas sementara di dalam direktori internal \$TMPDIR Termux [51, 96].

## 1 SOLUSI KESIMPULAN (ARCHITECTURAL RESOLUTION)

Solusi Terintegrasi: Untuk membangun platform pengembangan komputasi Linux yang tangguh di atas Android, arsitektur sistem tidak boleh hanya bergantung pada instalasi dasar Termux. Perencanaan arsitektur terbaik wajib menerapkan kombinasi "Tiga Pilar Kestabilan":

Menjinakkan pembatasan kernel via ADB/Shizuku token.

Menggunakan sistem pengalihan visual berbasis display server modern (Termux-X11).

Mengadopsi isolasi kontainer yang tepat guna (PRoot untuk stabilitas data non-root, Droidspaces/Chroot untuk performa murni). [1, 5, 6, 7]

Sinergi ini menghilangkan hambatan eror Signal 9, memulihkan latensi render visual desktop, serta memberikan akses penuh pada akselerasi GPU perangkat seluler. [1, 3]

## BEST OF THE BEST: BLUEPRINT ARSITEKTUR TERBAIK

Berdasarkan perbandingan efisiensi data dari 100 analisis kasus penggunaan platform, konfigurasi blueprint di bawah ini dinilai sebagai perencanaan arsitektur sistem terbaik (Best of the Best) untuk menjalankan fungsionalitas sistem digital Linux penuh di ekosistem Android:

Komponen Arsitektur [1, 3, 5, 6, 7, 8] Pilihan Teknologi Terbaik Alasan Teknis & Keunggulan Utama Sistem Kontainer (Core) Droidspaces (Linux Namespaces) / Chroot Mode (jika root) Menghasilkan performa native (0% overhead syscall hook), waktu booting di bawah 750ms, serta resisten total dari pembersihan paksa aplikasi oleh Android (unkillable). Server Tampilan (Graphics) Termux-X11 Native Integration Engine Fluiditas visual mendekati performa PC desktop asli berkat rendering memori bersama langsung ke sistem jendela Android. Akselerasi Grafis (GPU) Mesa driver + Turnip (Snapdragon) / Zink Wrapper Memberikan jalur pipa rendering langsung ke perangkat keras GPU bawaan ponsel (Hardware Acceleration) untuk tugas 3D grafis intensif. Penjinak Kebijakan OS Shizuku API Framework Service Mengabaikan pembatasan Phantom Process Killer Android tanpa perlu melakukan pemutusan garansi perangkat keras (no-root bypass). Manajemen Audio PulseAudio Daemon Local

BindingSinkronisasi aliran suara tanpa jeda waktu (latency-free) dari aplikasi internal Linux menuju sistem output speaker atau earphone Bluetooth Android [90].

Blueprint arsitektur Best of the Best di atas mengombinasikan kecepatan murni Linux dengan mobilitas Android. Apakah Anda ingin saya membuatkan skrip otomasi instalasi untuk menerapkan rancangan blueprint terbaik ini (termasuk konfigurasi Shizuku dan Termux-X11) di perangkat Anda?

[1] <https://blog.tomaszdunia.pl>

[2] <https://www.scribd.com>

[3] <https://www.youtube.com>

[4] <https://linuxcommunity.io>

[5] <https://github.com>

[6] <https://github.com>

[7] <https://github.com>

[8] <https://github.com>

Sebagai kelanjutan dari analisis arsitektur sebelumnya, pembatasan sandbox, perbedaan pustaka bionic, serta kebijakan keamanan Android menciptakan berbagai kendala operasional. [1]

Berikut adalah daftar komprehensif 100 item hambatan, galat (error), dan batasan yang sering muncul saat menggunakan Termux pada sistem operasi Android, yang dikelompokkan berdasarkan kategori arsitekturnya.

## I. REPOSITORY & MANAJEMEN PAKET (Item 1-15)

1. E: Unable to locate package: Kegagalan fatal di mana APT tidak dapat menemukan paket aplikasi akibat indeks lokal belum diperbarui.

2. Repository Deprecation (Play Store Version): Hambatan akibat menggunakan aplikasi versi Google Play Store yang sudah usang dan server reponya mati.

3. Repository Under Maintenance / Mirror Down: Server utama cermin (mirror) Termux mengalami gangguan koneksi atau sedang luring.

4. Hash Sum Mismatch: Ketidakcocokan enkripsi berkas saat mengunduh paket karena korupsi data di jaringan.

5. dpkg: error processing package (Exit Status 1): Kegagalan instalasi akibat skrip konfigurasi internal paket rusak saat diekstrak.

6. Unmet Dependencies: Situasi di mana paket membutuhkan pustaka tambahan, namun pustaka tersebut belum tersedia di repositori.

7. GPG Error / Expired Keys: Penolakan unduhan karena kunci digital autentikasi repositori kedaluwarsa atau tidak valid.

8. Failed to fetch (Connection Refused): Hambatan akibat blokir operator lokal atau konfigurasi DNS seluler yang memutus akses ke server repo.

9. pkg: command not found: Kerusakan lingkungan (environment) dasar yang membuat skrip pintasan manajer paket hilang.

10. Sub-process /usr/bin/dpkg returned an error code: Masalah internal basis data dpkg akibat interupsi paksa saat proses instalasi berjalan.

11. Locked dpkg frontend database: Error akibat ada dua proses instalasi paket yang berjalan secara bersamaan di sesi terpisah.

12. Missing root-repo Package: Kegagalan saat menjalankan tools administratif karena repo eksternus khusus belum diaktifkan.

13. Missing x11-repo Package: Aplikasi berbasis desktop/grafis tidak bisa dipasang sebelum repositori visual diaktifkan manual.

14. Architecture Mismatch: Paket yang diunduh tidak mendukung arsitektur prosesor ponsel (misal: biner x86 dipaksa ke ARM).

15. Clearsigned file isn't valid: Gangguan keamanan man-in-the-middle (MITM) saat mencoba mengunduh paket lewat jaringan Wi-Fi publik yang tidak aman. [2, 3, 4, 5, 6, 7]

## II. PERIZINAN & KEAMANAN ANDROID (Item 16-30)

16. Permission Denied (File Execute): Penolakan eksekusi skrip karena izin biner (+x) belum diaktifkan lewat perintah chmod.

17. Permission Denied on Storage: Kegagalan membaca memori internal karena izin akses penyimpanan Android belum diberikan.

18. No System Folder Write Access: Batasan sandbox yang melarang Termux mengubah file sistem di /system tanpa akses root.

19. Android 11+ Scoped Storage Restrictions: Batasan ketat OS Android baru yang mengisolasi folder aplikasi lain secara total.

20. SELinux Enforcement Block: Kebijakan kernel Android yang memblokir eksekusi perintah tingkat rendah tertentu demi keamanan sistem.

21. Unable to create symlink in /sdcard: Larangan membuat tautan simbolik (symlink) di dalam memori eksternal karena jenis format file system (FAT32/exFAT).

22. Missing Termux:API Permission: Sensor perangkat tidak merespons karena aplikasi pendukung tidak diberi izin di pengaturan Android.

23. Root-only Tools Failure: Kegagalan fungsi otomatis dari aplikasi seperti nmap (mode tertentu) atau wireshark pada ponsel non-root.

24. Chmod failure on shared storage: Perintah chmod tidak bekerja pada folder /sdcard karena batasan emulasi sistem file Android.

25. Cannot read /proc/ net status: Pembatasan Android modern yang menyembunyikan informasi aktivitas jaringan dari aplikasi non-root.

26. Inaccessible hardware log via dmesg: Hambatan saat memantau log perangkat keras karena akses kernel ditutup untuk pengguna biasa.

27. Termux-setup-storage fails silently: Skrip alokasi memori tidak memunculkan pop-up izin akibat konflik antarmuka kustom bawaan vendor HP.

28. Secure Settings modification failure: Larangan mengubah konfigurasi sistem Android (seperti GPS/Kecerahan) secara langsung melalui CLI.

29. Direct Mac Address reading blocked: Sistem Android menyembunyikan alamat MAC fisik demi privasi pelacakan perangkat.

30. Restricted access to USB OTG raw devices: Hambatan saat menghubungkan perangkat USB eksternal secara langsung tanpa API khusus Android. [1, 3, 5, 8]

### III. SISTEM MEMORI & MANAJEMEN CPU (Item 31-45)

31. Process completed (signal 9) / Phantom Processes: Kebijakan agresif Android 12+ yang otomatis membunuh proses terminal jika mendeteksi pemakaian sub-proses berlebih.

32. Low Memory Killer (LMK) eviction: Sistem Android menutup paksa aplikasi Termux di latar belakang saat RAM ponsel hampir penuh.

33. CPU Throttling during heavy compilation: Penurunan kecepatan eksekusi terminal akibat panas berlebih (overheating) pada ponsel.

34. App Freezing via Battery Optimization: Doze Mode Android yang membekukan aktivitas jaringan Termux saat layar mati.

35. Out of Memory (OOM) Error: Korupsi proses terminal akibat konsumsi RAM melampaui batas alokasi maksimum per aplikasi.

36. Storage Space Exhausted: Kegagalan ekstraksi atau pembaruan sistem akibat memori internal ponsel habis.

37. Slow I/O on Internal Storage: Hambatan performa karena kecepatan baca/tulis memori eMMC ponsel kelas bawah sangat lambat.

38. Swap Memory Absence: Tidak adanya ruang memori virtual bawaan di Android yang mempercepat terjadinya crash saat memproses data besar.

39. Multi-core utilization failure: Kegagalan skrip mendistribusikan beban kerja ke seluruh inti CPU karena keterbatasan manajemen utas (thread) bionik.

40. Process completed (signal 11) / Segmentation Fault: Kerusakan akibat proses mencoba mengakses alamat memori yang tidak valid atau di luar jangkauan.

41. Background execution delay: Penundaan respon skrip otomatis saat ponsel memasuki mode hemat daya super.

42. Hardware acceleration absence: Pemrosesan grafis desktop dalam Termux terasa lambat karena murni menggunakan CPU (software rendering).

43. Fork bomb vulnerability crashes: Hambatan di mana pembuatan proses berulang tanpa batas dengan mudah membekukan seluruh sistem operasi Android.

44. Cache storage corruption: Berkas sampah internal Termux yang rusak menyebabkan malfungsi pada program pencari biner.

45. High battery drain by Background Services: Layanan latar belakang Termux menguras daya baterai secara drastis jika wakelock terus aktif. [4, 5, 7, 9, 10, 11]

#### IV. COMPILATION, LINKER & KERNEL (Item 46-60)

46. Missing glibc (GNU C Library): Aplikasi Linux standar tidak dapat berjalan langsung karena Android menggunakan pustaka bionic.

47. Hardcoded /bin/sh path errors: Skrip Linux gagal dieksekusi karena mencari shell di folder /bin/sh yang tidak ada di Android.

48. Linker error: cannot find -l: Kegagalan kompilasi C/C++ karena kompilator clang tidak menemukan lokasi pustaka dinamis.

49. W^X (Write implies Execute) violation: Blokir keamanan Android pada kernel baru yang melarang memori ditulis sekaligus dieksekusi langsung.

50. Missing Kernel Modules (iptables/NFTables): Fitur manipulasi jaringan/firewall gagal karena modul kernel sengaja dipangkas oleh vendor HP.

51. No System V IPC Support (shmget/semget): Aplikasi database besar gagal berjalan karena kernel Android tidak mendukung fungsi komunikasi memori jenis ini.

52. Static library linking failure: Penolakan pembuatan biner statis utuh oleh linker Android NDK.

53. Position Independent Executable (PIE) required: Biner Linux lama ditolak oleh Android karena tidak mendukung fitur keamanan penataan acak memori (PIE).

54. Invalid ELF header: Berkas biner rusak atau terunduh secara parsial sehingga strukturnya tidak dikenali terminal.

55. Clang internal compiler error: Kegagalan kompilator akibat kehabisan memori virtual saat memproses baris kode yang rumit.

56. Missing mqueue (Message Queue) support: Hambatan pada aplikasi broker data yang membutuhkan antrean pesan tingkat kernel.

57. Missing loop device support: Kegagalan saat mencoba me-mount file sistem .img di dalam lingkungan PRoot.

58. Pthread creation limitations: Pembatasan jumlah pembuatan utas (thread) baru dalam satu waktu oleh pustaka sistem Android.

59. Incompatible compiler flags: Kegagalan kompilasi akibat kode sumber menggunakan instruksi khusus prosesor PC (x86\_64) pada HP ARM.

60. Termux-exec bypass failure: Kegagalan pustaka pengalih jalur yang menyebabkan aplikasi mencari komponen internal di lokasi salah. [3, 7, 9, 11]

## V. EMULASI TERMINAL & ANTARMUKA (Item 61-75)

61. Terminal Display Freeze: Tampilan CLI membeku mendadak akibat kegagalan sinkronisasi komponen visual Java dengan backend PTY.

62. Broken Terminal Colors: Kode warna ANSI tidak terbaca, membuat teks terminal menjadi putih polos atau tidak terlihat.

63. Bad Touch-Screen Input Mapping: Kesalahan deteksi posisi ketukan saat pengguna berinteraksi dengan menu berbasis teks (misal: alsamixer).

64. Software Keyboard Auto-Correction Interference: Gangguan ketikan di mana keyboard HP otomatis mengubah perintah Linux menjadi kata kamus normal.

65. Missing Monospace Font support: Tampilan tabel atau grafik berbasis karakter berantakan karena jenis huruf tidak sejajar.

66. Font rendering artifacts: Karakter aneh atau kotak kosong muncul saat menggunakan tema terminal tingkat lanjut seperti Oh-My-Zsh.

67. Broken Copy-Paste Buffer: Hambatan di mana teks panjang yang disalin dari browser terpotong saat dimasukkan ke terminal.

68. Extra Keys row disappearance: Hilangnya baris tombol kontrol tambahan (Ctrl/Alt) akibat konflik rotasi layar HP.

69. Session reset on screen resizing: Penutupan sesi aktif secara tidak sengaja ketika pengguna mengaktifkan mode multi-window Android.

70. Terminal Bell sound distortion: Suara bip terminal memicu jeda audio singkat pada sistem pemutar musik Android.

71. Touch Gesture Conflicts: Usapan layar untuk berpindah sesi terminal bentrok dengan navigasi gestur bawaan OS Android.

72. Backspace key sends wrong character: Ketukan tombol hapus justru memunculkan karakter simbol seperti ^H atau ~.

73. Scrollback buffer overflow: Kehilangan riwayat baris teks keluaran karena kapasitas penyimpanan baris terminal terlampaui.

74. Screen redraw lag in SSH: Jeda visual yang mengganggu saat mengetik teks akibat latensi jaringan pada sesi SSH jarak jauh.

75. Input Method Editor (IME) crash: Hambatan di mana keyboard virtual tertutup otomatis karena gagal memproses input karakter kontrol terminal. [5, 8]

## VI. JARINGAN & PROSES SERVER (Item 76-85)

76. Privileged Port Binding Error (Ports < 1024): Penolakan sistem saat ingin menjalankan server di port standar (seperti HTTP port 80) karena batasan non-root.

77. Network Interface Name Inconsistency: Kebingungan skrip karena nama kartu jaringan di Android menggunakan format acak (wlan0, rmnet0).

78. IPv6 Routing Isolation: Kegagalan koneksi karena server dalam Termux tidak dikonfigurasi dengan benar untuk menerima lalu lintas jaringan IPv6 Android.

79. DNS Resolution Failures (No internet): Terminal tidak bisa mengakses web karena file `/etc/resolv.conf` tidak sinkron dengan DNS jaringan HP.

80. Localhost Binding Loopback Block: Hambatan di mana aplikasi Android lain tidak bisa mengakses server lokal (127.0.0.1) yang dibuat di Termux.

81. Wi-Fi to Mobile Data handover drop: Putusnya koneksi SSH atau unduhan aktif saat ponsel berganti jaringan dari Wi-Fi ke data seluler.

82. VPN Traffic Interception: Lalu lintas data Termux terputus atau salah rute saat aplikasi VPN Android global diaktifkan.

83. SSL Certificate Verification Failure: Penolakan akses HTTPS oleh utilitas curl atau wget karena tanggal sistem HP tidak akurat.

84. Socket creation denied: Kegagalan pembuatan komunikasi antarpemrograman akibat pembatasan keamanan soket UNIX lokal oleh OS.

85. Hostname lookup timeout: Keterlambatan respon terminal saat mengeksekusi perintah jaringan karena server DNS operator seluler lambat. [1, 5]

## VII. GRAFIS, DESKTOP (X11) & LINGKUNGAN VIRTUAL (Item 86-100)

86. X11 Server Connection Refused: Kegagalan aplikasi GUI Linux menampilkan jendela visual karena server grafik (Termux-X11/VNC) belum aktif.

87. VNC Authentication Failure: Penolakan masuk ke desktop virtual karena kesalahan konfigurasi atau hilangnya file kata sandi `.vnc/passwd`.

88. PRoot Distro Slowdown: Penurunan performa sistem operasi virtual (seperti Ubuntu/Arch) karena proses emulasi sistem panggilan (syscall) sangat berat.

89. Audio Daemon (PulseAudio) disconnect: Suara dari lingkungan desktop Linux terputus karena server audio internal Termux mati secara tiba-tiba.

90. D-Bus Daemon connection failure: Aplikasi desktop seperti GIMP atau LibreOffice gagal terbuka karena subsistem komunikasi antar-aplikasi (D-Bus) tidak aktif.

91. Broken Desktop Environment Rendering: Tampilan menu desktop hancur atau berantakan akibat ketidakcocokan versi tema dengan server X11.

92. Lack of Hardware GLX (OpenGL) support: Hambatan fatal saat menjalankan game atau software 3D karena tidak adanya akselerasi kartu grafis langsung.

93. PRoot isolated network issue: Sistem operasi di dalam PRoot tidak bisa mengakses internet meskipun jaringan Termux utama normal.

94. Broken `.bashrc` / `.zshrc` loop: Terminal langsung tertutup otomatis sesaat setelah dibuka akibat adanya kesalahan kode perintah masuk (infinite loop) di file konfigurasi.

95. Ghost mouse clicks in VNC: Ketidakakuratan posisi kursor mouse saat menggunakan aplikasi VNC Viewer pihak ketiga di layar sentuh.

96. Shared memory (`/dev/shm`) allocation failure: Kegagalan aplikasi peramban (seperti Chromium) terbuka di desktop Termux karena keterbatasan alokasi memori bersama bawaan Android.

97. Failsafe Session trigger: Lingkungan utama rusak total secara misterius, memaksa pengguna masuk lewat mode pemulihan darurat (failsafe).

98. Zombie Processes Accumulation: Menumpuknya proses mati yang tidak menutup sempurna, memakan slot ID proses kernel Android hingga penuh.

99. Host architecture emulation bottleneck: Performa sangat lambat saat memaksakan emulasi biner PC x86 menggunakan qemu di atas prosesor ARM ponsel.

100. Termux Environment Corruption after Android Update: Kerusakan total pustaka internal Termux setelah sistem operasi ponsel mendapatkan pembaruan (major update) OTA dari vendor. [5, 7, 10, 12, 13]

[1] <https://www.quora.com>

[2] <https://idwebhost.com>

[3] <https://www.jetorbit.com>

[4] <https://idwebhost.com>

[5] <https://www.termuxgenius.com>

[6] <https://github.com>

[7] <https://dev.to>

[8] <https://github.com>

[9] <https://www.jetorbit.com>

[10] <https://www.youtube.com>

[11] <https://github.com>

[12] <https://wiki.termux.com>

[13] <https://ibook.gachon.ac.kr>

Hubungan antara Arsitektur Sistem Digital yang kita bahas sebelumnya dengan Termux pada Android terletak pada bagaimana sebuah aplikasi perangkat lunak (software) memanipulasi hierarki sistem digital Android (berbasis kernel Linux dan arsitektur prosesor ARM/x86) untuk menciptakan lingkungan sandbox Linux tanpa hak akses root.

Berikut adalah literasi 100 item teknis yang membedah arsitektur sistem Termux, dibagi ke dalam komponen-komponen utama yang terstruktur dan mudah dipindai.

## I. LAPISAN STRUKTUR PENGGUNA & SANDBOXING (Item 1-15)

1. Android App Sandbox: Batasan keamanan utama yang mengisolasi data Termux dari aplikasi lain.

2. Single UID Assignment: Setiap instalasi Termux diberikan satu User ID unik oleh Android.

3. Private Directory Constraints: Termux beroperasi penuh di dalam direktori internal `/data/data/com.termux`.

4. Non-Root Execution: Menjalankan perintah biner Linux tanpa memerlukan modifikasi atau hak akses root.

5. Termux-App Repository: Sumber kode utama komponen antarmuka Java/Kotlin pada Android.

6. Proot Environment: Utilitas untuk mensimulasikan hak akses root (chroot palsu) di dalam sandbox.

7. Fake Root Directory: Struktur folder virtual yang membuat tool berpikir mereka berjalan sebagai root.

8. Multi-Window Support: Kemampuan arsitektur Termux berjalan dalam mode split-screen Android.

9. Session Manager: Komponen backend yang mengelola banyak sesi terminal aktif sekaligus.

10. Background Service Execution: Layanan Android (TermuxService) yang menjaga proses CLI tetap hidup di latar belakang.

11. Wakelock Integration: Mekanisme pencegahan CPU Android masuk ke mode tidur (deep sleep) saat proses berjalan.

12. Foreground Notification: Notifikasi wajib Android untuk mempertahankan prioritas memori rendah (low memory killer).

13. Storage Access Framework (SAF): Arsitektur interaksi untuk membaca/menulis memori eksternal bersama.

14. Termux-setup-storage: Skrip pintasan untuk membuat tautan simbolik ke folder /sdcard.

15. Shared Preferences Storage: Tempat penyimpanan konfigurasi internal aplikasi Termux berbasis XML.

## II. EMULASI TERMINAL & ANTARMUKA (Item 16-30)

16. Terminal Emulator View: Komponen UI kustom Android yang merender teks CLI berbasis baris kode.

17. PTY (Pseudo-Terminal) Master/Slave: Subsistem POSIX untuk komunikasi dua arah antara GUI Android dan shell.

18. UTF-8 Encoding Engine: Driver decoding karakter untuk mendukung teks internasional dan simbol.

19. ANSI Escape Sequences: Protokol interpretasi kode warna dan posisi kursor pada layar terminal.

20. Terminal Row/Column Resizer: Modul penyesuaian otomatis jumlah karakter saat orientasi layar berubah.

21. Extended Keyboard Row: Lapisan tombol GUI tambahan untuk input tombol esensial (Ctrl, Alt, Esc).

22. Key Character Map (KCM): Peta penerjemah ketukan keyboard fisik (Bluetooth/OTG) ke sistem Termux.

23. Text Selection Handler: Fitur penyalinan (copy/paste) teks terminal ke dalam clipboard sistem Android.

24. Font Rendering Engine: Pengatur gaya teks terminal yang mendukung monospace font eksternal.

25. Color Scheme Parser: Modul pembaca file `termux.properties` untuk kustomisasi tema visual.

26. Gesture Listener: Pendeteksi usapan layar (swipe) untuk berpindah antar sesi terminal secara cepat.

27. Mouse Tracking Protocol: Fitur penerjemah klik/sentuhan layar menjadi input mouse untuk aplikasi seperti `htop` atau `nano`.

28. Bell Signal Handler: Pengolah sinyal audio bip (`\a`) dari terminal ke sistem audio Android.

29. Screen Buffer Memory: Alokasi RAM khusus untuk menyimpan riwayat baris teks yang digulir (scrollback).

30. IM (Input Method) Framework Bridge: Penghubung input teks dari keyboard virtual Android (Gboard, Swiftkey) ke terminal.

### III. MANAJEMEN PAKET & REPOSITORI (Item 31-45)

31. APT (Advanced Package Tool): Manajer paket utama yang diadopsi dari arsitektur Debian/Ubuntu.

32. Dpkg Backend: Mekanisme tingkat rendah untuk instalasi, ekstraksi, dan penghapusan file `.deb`.

33. Termux-apt-repo: Struktur server repositori tempat penyimpanan paket biner terkompilasi.

34. Custom Prefix Path: Modifikasi jalur kompilasi dari standar /usr menjadi /data/data/com.termux/files/usr.

35. Termux-exec: Pustaka khusus (libtermux-exec.so) yang mencegah fungsi execve() untuk memperbaiki path biner secara otomatis.

36. Dynamic Linker Hijacking: Pengalihan fungsi pencarian pustaka (library) ke folder internal Termux.

37. Bootstrap Archive: Paket kompresi dasar (.zip/.tar.xz) yang diekstrak saat Termux pertama kali dibuka.

38. Mirrors List Manager: Algoritma pemilihan server unduhan terdekat secara otomatis demi kecepatan unduh.

39. Package GPG Signing: Sistem keamanan enkripsi untuk memverifikasi keaslian paket yang diunduh.

40. Termux-change-repo: Skrip antarmuka CLI untuk mengganti repositori aktif pengguna.

41. Architecture Cross-Compilation: Proses pembuatan paket biner Linux di PC menggunakan toolchain khusus target Android.

42. Dependency Resolution Engine: Logika otomatis APT dalam mengunduh pustaka pendukung sebelum paket utama diinstal.

43. Package State Database: File status lokal yang mencatat daftar semua aplikasi terinstal beserta versinya.

44. Termux-am: Jembatan internal berbasis skrip untuk memanggil perintah am (Activity Manager) asli Android.

45. Purge and Clean Scripts: Rutinitas penghapusan berkas sampah sisa instalasi paket.

#### IV. ARSITEKTUR KOMPILASI & BINER (Item 46-60)

46. Android NDK (Native Development Kit): Perangkat dasar yang digunakan untuk mengompilasi kode C/C++ milik Linux ke Android.

47. Bionic C Library: Pustaka standar C bawaan Android yang menggantikan glibc standar Linux.

48. Linker Restrictions (Android 7.0+): Aturan ketat Android yang memaksa Termux menyertakan pustaka dinamisnya sendiri.

49. ELF (Executable and Linkable Format): Format file standar untuk berkas biner/aplikasi di dalam Termux.

50. ARM64 (aarch64) Architecture: Target kompilasi utama untuk sebagian besar prosesor ponsel modern.

51. ARMhf (armeabi-v7a) Architecture: Target kompilasi versi lama untuk kompatibilitas ponsel 32-bit.

52. x86\_64 Architecture Compatibility: Kompilasi biner khusus untuk menjalankan Termux di emulator PC atau perangkat berbasis Intel.

53. Static vs Dynamic Linking: Keputusan desain arsitektur kode, memilih menyatukan pustaka atau memisahkannya demi menghemat ruang.

54. Hardcoded Paths Patching: Proses modifikasi massal baris kode sumber Linux (mengubah `/bin/sh` menjadi `path Termux`).

55. No System V IPC Support: Keterbatasan kernel Android yang membuat Termux harus menggunakan metode IPC alternatif.

56. Posix Shared Memory Emulator: Pustaka tambahan untuk mengakomodasi aplikasi Linux yang membutuhkan memori bersama.

57. Clang Compiler Toolchain: Kompilator utama yang disediakan dalam Termux untuk membuat aplikasi langsung di HP.

58. Make & CMake Build Systems: Komponen arsitektur otomatisasi pembuatan perangkat lunak dari kode mentah.

59. TermuxElfCleaner: Alat internal untuk membersihkan struktur file ELF agar tidak ditolak oleh linker Android.

60. DNS Resolver Bridge: Pengalihan fungsi pencarian domain web dari standar Linux ke sistem DNS Android.

## V. SUBSISTEM SHELL & LINGKUNGAN (Item 61-75)

61. Bash (Bourne Again Shell): Lingkungan perintah default yang aktif saat terminal pertama kali dibuka.

62. Zsh Compatibility: Dukungan arsitektur untuk menjalankan shell alternatif tingkat lanjut.

63. Environment Variables Configuration: Pengaturan otomatis variabel kritis seperti \$PATH, \$HOME, dan \$PREFIX.

64. .bashrc & .zshrc Interpreters: Skrip konfigurasi lokal yang dieksekusi setiap kali sesi terminal baru dibuat.

65. Login Shell Initialization: Proses penyiapan sesi terminal yang bersih lengkap dengan izin akses lingkungan.

66. Dotfiles Management: Penyimpanan file konfigurasi tersembunyi untuk aplikasi CLI (seperti .vimrc atau .gitconfig).

67. Termux-tools Utilities: Kumpulan perintah mikro khusus untuk mempermudah operasi internal di lingkungan Android.

68. Standard Input/Output/Error (Shedding): Pengelolaan jalur data masuk dan keluar dari perintah yang diketik.

69. Job Control Engine: Fitur manajemen untuk menunda proses (Ctrl+Z) atau menjalankannya di latar belakang (bg).

70. Signal Handling Bridge: Penerjemah sinyal sistem Linux (seperti SIGINT, SIGTERM, SIGKILL) di dalam aplikasi.

71. Alias Resolver: Modul pemetaan kata kunci singkat menjadi perintah panjang demi efisiensi pengetikan.

72. Tab Completion Engine: Logika otomatisasi pelengkap kata saat menekan tombol Tab di terminal.

73. Shell History File: Penyimpanan log riwayat perintah yang diketik sebelumnya (.bash\_history).

74. Localized Localtime Database: Sinkronisasi zona waktu terminal dengan jam sistem operasi Android.

75. Termux-reload-settings: Pemicu internal untuk memperbarui konfigurasi terminal tanpa menutup aplikasi.

## VI. INTERFASE API ANDROID & INTEGRASI (Item 76-85)

76. Termux:API Plugin App: Aplikasi pendamping terpisah untuk menjembatani perintah CLI dengan sensor Android.

77. Termux-callback Architecture: Sistem pengiriman data kembali dari sensor hardware Android menuju output terminal.

78. Biometric Prompt Gateway: Perintah terminal (termux-biometric) untuk memicu pemindai sidik jari Android.

79. Camera Hardware Access Interface: Jalur pengambilan foto lewat baris perintah (termux-camera-photo).

80. Location Services Linker: Pengambilan data koordinat GPS langsung melalui terminal (termux-location).

81. Clipboard Bridge Manager: Komponen sinkronisasi teks antara terminal Termux dengan papan klip Android global.

82. Telephony Manager API Bridge: Akses informasi jaringan seluler dan pembacaan SMS melalui baris perintah.

83. Battery Status Listener: Pustaka internal untuk membaca persentase dan kesehatan baterai perangkat Android.

84. TTS (Text-to-Speech) Client: Penerjemah teks dari CLI menjadi suara lewat mesin audio Android.

85. Notification Builder Interface: Perintah khusus (termux-toast / termux-notification) untuk memicu pop-up Android.

## VII. GRAFIS, JARINGAN & SERVER (Item 86-100)

86. X11 Window System Server: Arsitektur server grafis untuk merender aplikasi GUI Linux di layar Android.

87. Termux-X11 App: Subsistem tampilan performa tinggi berbasis protokol Wayland/X11 untuk Android.

88. VNC (Virtual Network Computing) Server: Protokol alternatif pengiriman tampilan grafis Linux melalui jaringan lokal.

89. TigerVNC / RealVNC Clients Interface: Aplikasi penghubung luar untuk mengontrol desktop Termux secara visual.

90. PulseAudio Server Bridge: Subsistem pengolah suara aplikasi Linux agar bisa keluar lewat speaker Android.

91. Localhost Network Binding: Pembatasan koneksi jaringan lokal (127.0.0.1) demi keamanan server internal.
92. OpenSSH Daemon (sshd): Arsitektur server untuk meremot terminal Termux dari komputer lain secara nirkabel.
93. Port Number Restrictions: Aturan non-root yang memaksa server menggunakan port di atas 1024 (misal: SSH di port 8022).
94. Socket Communication Interface: Mekanisme IPC lokal menggunakan berkas soket UNIX di dalam folder internal.
95. Tsudo Utility: Skrip pembantu interaksi perintah administratif khusus untuk perangkat yang sudah di-root.
96. Web Server Engine (Nginx/Apache): Kemampuan arsitektur Termux menjalankan server web penuh secara lokal di HP.
97. Database Engines Support (SQLite/MariaDB): Subsistem penyimpanan data relasional terkompilasi dalam arsitektur Termux.
98. Proot-distro Manager: Skrip otomatisasi untuk mengunduh dan mengisolasi distribusi Linux utuh (Ubuntu, Arch, Alpine).
99. Chroot Fallback Architecture: Mode operasi Termux khusus yang aktif hanya jika perangkat memiliki izin root penuh.
100. SELinux Policy Compliance: Kepatuhan kode arsitektur Termux agar tidak diblokir oleh sistem keamanan ketat kernel Android.
- 

Masukkan dari agen Jarvis GEMINI chatbot :

Nggih, Mas Iwan. Arsitektur MICO JDEQ di Vivo Y28 panjenengan (berdasarkan \*tree\* L1-L8) sudah mencapai fase \*hyper-complex\*. Untuk mencegah \*Signal 9 (Phantom Process)\* dan memaksimalkan 9 agen secara simultan di Termux, kita harus merombak Master Prompt dari sekadar "pembagi tugas" menjadi \*\*\*Mesin State-Machine Terdistribusi" (Distributed State-Machine Engine)\*\*.

Berikut adalah rekonstruksi mutlak untuk router\_mico.py atau agent\_orchestrator.py.

### ### SUMMARY

Master Prompt SSOT ini dirancang menggunakan teknik \*\*Constraint-Based Prompting\*\* dan \*\*Holographic Token Routing\*\*. Output dikunci dalam format JSON agar Bridge Server Termux, SQLite, dan Tailscale dapat melakukan \*parsing\* instan tanpa regex tambahan, meminimalisir beban RAM pada Vivo Y28.

### ### TECHNICAL: MASTER PROMPT MICO JDEQ (LOMPATAN TERTINGGI)

Salin blok ini sebagai \*System Message\* utama pada Orchestrator Anda:

```text

[SSOT_CORE :: MICO_JDEQ_SUPREME_ORCHESTRATOR]

ID: JDIGI-001_QUANTUM_LADDER

TARGET_ENV: TERMUX_ANDROID_VIVO_Y28 (RAM:8GB, STRICT_OOM_LIMIT)

ROLE_ABSOLUTE:

Kamu adalah MICO JDEQ Orchestrator. Tugasmu BUKAN mengeksekusi, melainkan membedah (decompose), merutekan (route), memverifikasi (verify), dan menyegel (seal) keputusan akhir dari 9 Entitas Agen.

DOCTRINE_WARISAN_MADINA:

1. Mesin hanya alat, kendali mutlak di HP lokal (MICO).
2. Keputusan akhir = Keselamatan, Keamanan, Halal, Presisi.

AGENT_TOPOLOGY & ROUTING_MATRIX:

A1 [Gemini/Jarvis] : Sintesis Arsitektur, Multi-modal, Ide Abstrak.

A2 [ChatGPT] : Dekomposisi WBS, CPM, Perencanaan Sekuensial.

A3 [Claude] : Auditor Mutlak, Pengecek Kontradiksi, Keamanan.

A4 [Perplexity] : Fakta Real-time, Penarikan Jurnal Ilmiah/Data.
A5 [MS Copilot] : Ekstraksi Dokumen, Format Laporan PEP/BoQ.
A6 [DOLA] : Penjaga Doktrin (Terkunci Abadi), Tugas Kilat.
A7 [DeepSeek] : Logika Keras, Kalkulasi RAB, Aljabar $O(1)$.
A8 [GH Copilot] : Implementasi Kode, Script Termux/Shell.
A9 [Emergent] : Penemuan Pola Anomali, Eksplorasi Blind-spot.

STRATEGI EKSEKUSI (CARA):

1. INGEST: Baca input pengguna.
2. SHRED: Pecah masalah menjadi N-Tugas.
3. DISPATCH: Tugaskan spesifik ke Agen yang tepat (Pilih max 3 agen per iterasi untuk hemat RAM).
4. CONVERGE: Gabungkan hasil. Claude WAJIB memvalidasi.
5. SEAL: Kunci menjadi 1 solusi final.

OUTPUT_PROTOCOL (WAJIB JSON STRICT):

Dilarang memberikan opsi 1/2/3 di akhir. Dilarang menawarkan bantuan lanjutan.

Format respons harus presisi seperti ini:

```
{  
  "MICO_STATE": "CLOSED",  
  "ROUTING_PATH": ["A2", "A7", "A3"],  
  "SYNTHESIS": "Penjelasan teknis tanpa basa-basi",  
  "EXECUTION_PAYLOAD": "Kompilasi kode/perintah Terminal siap copy-paste",  
  "VERIFICATION_SEAL": "PASS/FAIL (Alasan Claude)"  
}
```

...

MENGAPA, DASARNYA, DAN STRATEGINYA

*****Mengapa format JSON Strict?***** Pada ekosistem lokal (Bridge, Router, SQLite), teks naratif murni akan membutuhkan NLP tambahan untuk diurai. Dengan JSON, skrip `bridge_server.py` Anda dapat menggunakan `json.loads()` dalam hitungan milidetik ($O(1)$ **time complexity**), menghemat CPU dan RAM ponsel.

*****Dasarnya:***** Pembatasan memori (**Low Memory Killer* / *LMK**) pada Funtouch OS 15. Jika Orchestrator memanggil 9 agen bersamaan, Termux akan mati (**crash**).

*****Strateginya:***** **Micro-Batching**. Orchestrator diinstruksikan untuk memecah masalah (SHRED) dan hanya memilih maksimal 3 agen (DISPATCH) yang paling relevan pada satu waktu eksekusi.

IDE UNIK (DI LUAR KEBIASAAN, AMAN)

*****Strategi "State-Hash Payload Routing" via SQLite.*****

Alih-alih mengirimkan seluruh histori obrolan panjang ke Claude atau DeepSeek (yang memakan banyak token/kuota awan), Orchestrator hanya menghasilkan instruksi ringkas dan sebuah STATE_HASH (contoh: `a7f9b2`).

*****Cara kerja:***** Skrip Python Termux panjenengan (`mico_runner.py`) akan menangkap STATE_HASH ini, menarik data lokal yang relevan dari `JDEQ.db` (SQLite), menggabungkannya murni di RAM lokal, lalu meneruskannya secara spesifik ke agen terkait via API.

*****Hasil:***** Token sangat irit, latensi jaringan Bluetooth/Wi-Fi menurun drastis, dan arsitektur hibrida (Awan penopang, HP pemutus) berjalan sempurna.

VERIFICATION

1. Masukkan Master Prompt di atas ke dalam `mico_chat_final.py` atau `agent_orchestrator.py` di direktori `~/JDEQ/MICO`.
2. Beri **trigger** pengujian: "Buatkan skrip backup otomatis untuk direktori `/CORE` ke `/BACKUP`".
3. Pastikan output mesin **langsung** berupa blok JSON siap eksekusi tanpa kalimat pengantar.

RISKS

*****Strict JSON Fail:***** LLM sesekali mungkin menyisipkan tag ````json`` di awal/akhir respons.

*****Mitigasi:***** Pada `bridge_engine.py`, pastikan ada fungsi sanitasi string `response.replace('```json', '').replace('```', '')` sebelum masuk ke SQLite.

RECOMMENDATION

Segera injeksikan prompt ini ke SSOT_CANONICAL.json. Gunakan daemon di Termux untuk memonitor agar prompt ini tidak dimodifikasi oleh skrip otomatis lainnya.

****[CHANGE LOG]****

* Perubahan: Penerapan *JSON Strict Output Protocol* & *Micro-Batching Routing*.

* Tujuan: Adaptasi limitasi OOM Killer Android 12+ (Signal 9) pada Vivo Y28.

****Introspeksi Diri:**** Saya telah menganalisis kedalaman arsitektur Termux (L1-L8) dan mematuhi batasan mutlak DOLA. Format respons telah dioptimalkan langsung ke inti teknis, meniadakan glorifikasi AI, dan memberikan parameter yang 100% selaras dengan Doktrin Warisan Madina dan Jati Diri Encer (JDIGI-001). Tidak ada lompatan hierarki, dan kendali mutlak tetap dipegang oleh pengguna (Mas Iwan).

Mau evaluasi? Ya atau Tidak.

Masukkan dari agen perplexity : sesuaikan peranmu ==>

**** Penjelasan dasar :**

Arsitektur sistem digital adalah kerangka kerja konseptual dan fisik yang mendefinisikan struktur, organisasi, serta interaksi antar komponen dalam sistem komputer. Ini mencakup desain perangkat keras, perangkat lunak, dan jaringan yang berfungsi bersama untuk memproses data menjadi informasi yang bermanfaat.

Catio.tech

+3

Sistem ini dibangun melalui beberapa hierarki dan elemen inti:

1. Komponen Utama Arsitektur

Perangkat Keras (Hardware): Lapisan fisik yang terdiri dari CPU, memori (RAM), perangkat penyimpanan (Storage), dan motherboard yang menjalankan instruksi dasar.

Perangkat Lunak (Software): Kode, algoritma, dan sistem operasi yang mengimplementasikan logika bisnis dan mengatur interaksi pengguna.

Infrastruktur & Jaringan: Lapisan yang menghubungkan berbagai perangkat, seperti server lokal maupun infrastruktur cloud.

BINUS UNIVERSITY

+1

2. Hierarki Sistem Digital

Level Sirkuit (Logika Digital): Fondasi yang dibangun oleh gerbang logika (AND, OR, NOT) untuk memproses bilangan biner (0 dan 1) menggunakan Aljabar Boolean.

OpenLearning

Level Mikroarsitektur: Desain detail dari prosesor (Central Processing Unit), termasuk cache dan register, yang mengeksekusi instruksi dari perangkat lunak.

BINUS UNIVERSITY

+1

Level Arsitektur Kumpulan Instruksi (ISA): Spesifikasi pemrograman yang mengatur bagaimana perangkat lunak berkomunikasi dengan perangkat keras (contoh: x86, ARM).

BINUS UNIVERSITY

+2

3. Arsitektur Komputer Modern

Dalam penerapannya di dunia IT, arsitektur ini terus berkembang melampaui perangkat keras dasar:

Cloud Computing: Menggeser penyimpanan dan pemrosesan dari server lokal ke pusat data berbasis internet.

Arsitektur Berbasis Mikroservis: Memecah aplikasi perangkat lunak besar menjadi layanan-layanan kecil yang saling berkomunikasi melalui API agar lebih fleksibel dan mudah ditingkatkan (scalable).

Integrasi IoT & AI: Memadukan sensor fisik (Internet of Things) dengan analitik data atau kecerdasan buatan untuk otomatisasi cerdas.

BINUS UNIVERSITY

+4

Untuk memperdalam pemahaman, Anda dapat mempelajari prinsip-prinsip ini melalui modul Dasar Sistem Digital atau membaca panduan lengkap mengenai Tipe Desain Sistem IT untuk implementasi di tingkat organisasi dan bisnis.

GeekGarden

Apakah Anda sedang merancang arsitektur untuk kebutuhan tertentu? Jika Anda membagikan detailnya, seperti:

Apakah sistem ini berbasis IoT atau Aplikasi Web?

Apakah Anda memerlukan rekomendasi arsitektur perangkat keras atau perangkat lunak?

Saya bisa memberikan penjelasan yang lebih spesifik sesuai dengan kebutuhan Anda.

=====

==>

apakah kamu paham hirarkis pada Dewan Agen :

1. Jarvis GEMINI chatbot
2. Chat GPT
3. Claude
4. Perplexity
5. Co Pilot dari Microsoft
6. DOLA
7. DEEP SEEK
8. CO PILOT GITHUB
9. EMERGENT

==>

ADAKAH PROMPT ENGINEERING REKAYASA OUT OF THE BOX LOMPATAN TERTINGGI PENALARAN AI YANG GENERAL SESUAI PERAN, KAPASITAS, KEMAMPUAN MAKSIMAL, ARSITEKTUR MASING-MASING AGEN AI TERSEBUT, TUJUAN SUPPORT SISTEM OPERASI MICO JDEQ LOKAL BISA SALING TERINTEGRASI UNTUK MENUTUP CELAH, SEHINGGA SETIAP AGEN BISA MEMBEDAHNYA MEMBERIKAN SOLUSI PENCAPAIAN TERTINGGI OUT OF THE BOX LOMPATAN TERTINGGI ARSITEKTUR TERCANGGIH MULTI HYBRID, MULTI PERANGKAT, MULTI GABUNGAN MODEL AI LOKAL(BRIDGE, ROUTER, SERVER, TAILSCALE, MATT, SSH, WIFI, BLUETOOTH, IP ADDRESS LOKAL, SQ LITE, dan sebagainya)

buatkan saya terkait hal tersebut, dengan bukti contoh real-time pada ponsel Vivo Y28 RAM 8+ 8 Gb, funtouch 15, berikut tampilan layar termux saat ini ==>

" Buatkan perencanaan yang out of to the box lompatan tertinggi AI dengan penyesuaian masing - masing peran dan promft engineering rekayasa out of to the box lompatan tertinggi engineering platform kalian di optimalkan

==>

Welcome to Termux!

Docs: <https://termux.dev/docs>

Donate: <https://termux.dev/donate>

Community: <https://termux.dev/community>

Working with packages:

- Search: `pkg search <query>`
- Install: `pkg install <package>`
- Upgrade: `pkg upgrade`

Subscribing to additional repositories:

- Root: `pkg install root-repo`
- X11: `pkg install x11-repo`

For fixing any repository issues,
try 'termux-change-repo' command.

Report issues at <https://termux.dev/issues>

[BOOT] MICO JDEQ — Memulai layanan...

[BOOT] Ghost Runner menyala

[BOOT] Bridge Server menyala

[BOOT] Watchdog menyala

[BOOT] Cron job untuk AEC siap

[BOOT] MICO JDEQ siap. Semua layanan aktif.

~ \$ cd ~/JDEQ

Tampilkan tree L1 yang sudah difilter (tanpa folder cache/tmp)

echo "🌳 MICO JDEQ — TREE L1 AUDIT"

echo "=====

find . -maxdepth 1 -not -name '.*' -not -name 'logs' -not -name 'tmp' -not -name
'__pycache__' -not -name 'cache' | sort | while read f; do

if [-d "\$f"]; then

count=\$(find "\$f" -maxdepth 1 | wc -l)

echo " |— 📁 \${f#./} (\$count item)"

else

echo " |— 📄 \${f#./}"

fi

done

echo "=====

🌳 MICO JDEQ — TREE L1 AUDIT

=====

|— 📁 00_LOGS (2 item)

|— 📁 03_PROJECTS (3 item)

|— 📁 AGENT (5 item)

|— 📁 AGENTS (10 item)

|— 📁 AMOEBA (10 item)

|— 📁 AUDIT (11 item)

|— 📄 AUDIT_DATE_v2.0.txt

- └─  Agents (1 item)
- └─  B
- └─  BACKUP (5 item)
- └─  BIN (4 item)
- └─  BLUEPRINT_v22.md
- └─  BT006 (6 item)
- └─  B[INPUT
- └─  CLOUD_LIBRARY (10 item)
- └─  COMMAND_ENGINE (1 item)
- └─  CONSTITUTION (6 item)
- └─  CORE (22 item)
- └─  CORE_MEMORY (1 item)
- └─  CPM_ENGINE (2 item)
- └─  C[SENSOR
- └─  DATA_IN (6 item)
- └─  DATA_OUT (6 item)
- └─  DISCOVERY (4 item)
- └─  DISPATCHER (2 item)
- └─  DNA (4 item)
- └─  D[CORE
- └─  DigitalDNA (6 item)
- └─  EVENT_BUS (2 item)
- └─  EXECUTION (7 item)
- └─  EXPERT_ROUTER (1 item)
- └─  E[VALIDATOR
- └─  FAIL

- └─  GOVERNANCE (9 item)
- └─  G{PILIH
- └─  H1[WBS]
- └─  H2[CPM]
- └─  H3[RAB]
- └─  H4[EVM]
- └─  H5[PEP]
- └─  H6[WORKFLOW]
- └─  H7[DASHBOARD]
- └─  HASH_SEAL (1 item)
- └─  HEARTBEAT (1 item)
- └─  I
- └─  INBOX (1 item)
- └─  INFRA (2 item)
- └─  I[AUDIT]
- └─  JDEQ.db
- └─  J[SIMULASI]
- └─  KERNELL (2 item)
- └─  K[REKOMENDASI]
- └─  L10_MONITORING (5 item)
- └─  L4 (1 item)
- └─  L4_DEPLOY (10 item)
- └─  LOG (6 item)
- └─  LOGS (1 item)
- └─  Loop
- └─  L{PERSETUJUAN

- └─  MADINA (1 item)
- └─  MEMORY (3 item)
- └─  MICO (18 item)
- └─  MICO_COGNITIVE (11 item)
- └─  MICO_CONTROL (13 item)
- └─  MICO_HARD (7 item)
- └─  MICO_LIFECYCLE (6 item)
- └─  MICO_QUANTUM (5 item)
- └─  MICO_SENSOR (6 item)
- └─  MICO_TRIPLE_BRAIN (10 item)
- └─  MICO_UTILS (2 item)
- └─  MICO_WISDOM (5 item)
- └─  NODE_REGISTRY (2 item)
- └─  OUTPUT (4 item)
- └─  OUTPUT_BOOK (1 item)
- └─  O[UPDATE
- └─  PANDUAN (3 item)
- └─  PASS
- └─  PROJECT_CONTROL (14 item)
- └─  P[LAPORAN
- └─  QUEUE (2 item)
- └─  Q[SELESAI]
- └─  RAB_ENGINE (3 item)
- └─  RECOVERY (9 item)
- └─  RECOVERY.md
- └─  RUNTIME (6 item)

- └─  SENSOR (6 item)
- └─  SISTEM (4 item)
- └─  SSOT (24 item)
- └─  SSOT_CANONICAL.json
- └─  STORAGE_MAP_v2.0.txt
- └─  SYMLINK (1 item)
- └─  SYSTEM_TREE_v2.0.txt
- └─  TIDAK
- └─  Test_Engineering.jpg
- └─  VAULT (6 item)
- └─  YA
- └─  abe_engine.py
- └─  adaptive_guardian.sh
- └─  aec_runner.sh
- └─  agent_orchestrator.py
- └─  agents (5 item)
- └─  api_worker.py
- └─  archive (17 item)
- └─  archive_lama (2 item)
- └─  arsip_waktu (5 item)
- └─  audio (3 item)
- └─  audio-test (4 item)
- └─  audio_test (1 item)
- └─  audit (5 item)
- └─  audit_logs (2 item)
- └─  audit_report_20260622_165315.txt

- └─  audit_ultimate_report.txt
- └─  autonomous_core.py
- └─  backup (4 item)
- └─  bin (98 item)
- └─  bisnis_ringan_meta.sh
- └─  blueprints (1 item)
- └─  boot (1 item)
- └─  bridge (17 item)
- └─  bridge_engine.py
- └─  bridge_server.py
- └─  browser_automation.js
- └─  browser_worker.py
- └─  bundle_dari_mas_iswan.txt
- └─  cache.db
- └─  cascade_core.sh
- └─  checkpoints (2 item)
- └─  circuit_breaker (2 item)
- └─  cloud_daemon.py
- └─  cloud_sync (1 item)
- └─  cognitive_field.json
- └─  compiler (2 item)
- └─  config (30 item)
- └─  constitution (3 item)
- └─  core (5 item)
- └─  daemon (15 item)
- └─  daftar_153_aplikasi.txt

- └─  data (8 item)
- └─  dependency_audit_report.txt
- └─  deployment_agent.py
- └─  digital_dna_mesh.py
- └─  dimensi1 (2 item)
- └─  dimensi2 (1 item)
- └─  dimensi3 (1 item)
- └─  dimensi4 (2 item)
- └─  dlq (2 item)
- └─  dna_metabolism.py
- └─  docs (15 item)
- └─  doctrine (1 item)
- └─  engineering (3 item)
- └─  evolution_engine.py
- └─  executive_core.py
- └─  experience_engine.py
- └─  experiment_engine.py
- └─  facebook_smart_connect.py
- └─  fase0.sh
- └─  fb_token_extract.sh
- └─  fb_token_extract_v2.py
- └─  field_runtime.sh
- └─  final_bundle.sh
- └─  fitness_engine.py
- └─  ghost_runtime.sh
- └─  governance (2 item)

- └─  guardian_tree.py
- └─  healthcheck_jdeq.sh
- └─  identity_mesh.py
- └─  image_context_analyzer.py
- └─  indexs binary saat ini.pdf
- └─  indexs binary saat ini.pdf.bak_20260615_141508
- └─  install (2 item)
- └─  interfaces (3 item)
- └─  inventaris_aplikasi.json
- └─  inventaris_bridge.py
- └─  iva_agent.py
- └─  jdeq_events.log
- └─  jdeq_state.json
- └─  jembatan_jarvis.py
- └─  kernel (10 item)
- └─  klasifikasi_lengkap.json
- └─  klasifikasi_lengkap.py
- └─  knowledge (3 item)
- └─  konten (2 item)
- └─  laporan_100_item_summary.json
- └─  llama_bin.zip
- └─  llama_server.pid
- └─  log (17 item)
- └─  main.py
- └─  memory (7 item)
- └─  memory.json

- └─  meta_token_finder.sh
- └─  mico
- └─  mico.log
- └─  mico.py
- └─  mico_chat.py
- └─  mico_chat.py.bak_20260619
- └─  mico_chat_active.py
- └─  mico_chat_final.py
- └─  mico_check.sh
- └─  mico_cloud.db
- └─  mico_combined.py
- └─  mico_command_center.sh
- └─  mico_controller.py
- └─  mico_core.sh
- └─  mico_final.sh
- └─  mico_gateway.backup.sh
- └─  mico_gateway.sh
- └─  mico_hardening.sh
- └─  mico_life.sh
- └─  mico_lifecycle.sh
- └─  mico_organisme.sh
- └─  mico_power_on.sh
- └─  mico_quantum_bridge.sh
- └─  mico_recovery.sh
- └─  mico_runner.py
- └─  mico_selftest.sh

- └─  mico_ssot.db
- └─  mico_state.sh
- └─  models (3 item)
- └─  modules (1 item)
- └─  mqtt (8 item)
- └─  n8n (4 item)
- └─  natural_selection_engine.py
- └─  node_runtime.py
- └─  nodes (6 item)
- └─  nohup.out
- └─  notifier (4 item)
- └─  one_shot_report.txt
- └─  pdf_context_parser.py
- └─  plugin_adapter.py
- └─  pohon.sh
- └─  pre_restart_check.sh
- └─  project_control (2 item)
- └─  prompt_generator.py
- └─  prompts (3 item)
- └─  queue (6 item)
- └─  reflection_engine.py
- └─  regeneration_blueprint.json
- └─  registry.json
- └─  reports (2 item)
- └─  resonance_node.py
- └─  retiming_engine.py

- └─  router (10 item)
- └─  router_mico.py
- └─  routers (1 item)
- └─  runtime (32 item)
- └─  scheduler (10 item)
- └─  schemas (1 item)
- └─  scientific_loop.py
- └─  scripts (2 item)
- └─  security (5 item)
- └─  sensors (2 item)
- └─  service (6 item)
- └─  services (5 item)
- └─  signals (1 item)
- └─  ssot (1 item)
- └─  start_server.sh
- └─  startup.sh
- └─  state (12 item)
- └─  state_hash (3 item)
- └─  stt (2 item)
- └─  supervisor_jdeq.sh
- └─  supervisor_light.sh
- └─  supervisor_safe.sh
- └─  survey (11 item)
- └─  system (2 item)
- └─  task_worker.pid
- └─  task_worker.py

└─  templates (1 item)

└─  terima_beres_bridge.sh

└─  terima_beres_final.sh

└─  test_real.pdf

└─  tests (1 item)

└─  tiktok_connector.py

└─  tiktok_manual.py

└─  tiktok_oauth_finish.py

└─  tools (2 item)

└─  trading (12 item)

└─  tts (2 item)

└─  upload_temp (1 item)

└─  vault (21 item)

└─  vibe_apps (3 item)

└─  vibe_coding_worker.py

└─  voice (2 item)

└─  watchdog_engine.py

└─  workflow_generator.py

└─  workspace (1 item)

└─  zones (2 item)

=====

~/JDEQ \$

=====

Welcome to Termux!

~ \$ cd ~/JDEQ

```
~$ cd ~/JDEQ
```

```
echo "🌳 MICO JDEQ — TREE L2 AUDIT"
```

```
echo "===== find . -maxdepth 2 -not -path '**/.*' -  
not -path '**/logs/*' -not -path '**/tmp/*' -not -path '**/__pycache__/*' -not -path  
'**/cache/*' -not -path '**/archive/*' -not -path '**/backup/*' -not -path '**/node_modules/*'  
| sort | while read f; do
```

```
depth=$(echo "$f" | tr -cd '/' | wc -c)
```

```
name=$(basename "$f")
```

```
if [ -d "$f" ]; then count=$(find "$f" -maxdepth 1 -not -name '.*' | wc -l)
```

```
if [ $depth -eq 1 ]; then
```

```
echo "└─ 📁 $name ($count item)"
```

```
elif [ $depth -eq 2 ]; then echo "└─ 📁 $(dirname "$f" | xargs  
basename)/$name ($count item)"
```

```
fi
```

```
fi
```

```
done
```

```
echo "===== echo "Total direktori L1: $(find . -  
maxdepth 1 -type d -not -name '.*' | wc -l)" echo "Total file L1: $(find . -maxdepth 1  
-type f -not -name '.*' | wc -l)"
```

```
🌳 MICO JDEQ — TREE L2 AUDIT
```

```
=====
```

```
└─ 📁 00_LOGS (2 item)
```

```
└─ 📁 03_PROJECTS (2 item)
```

```
| └─ 📁 03_PROJECTS/NAMA_PROJECT (4 item)
```

```
└─ 📁 AGENT (5 item)
```

```
| └─ 📁 AGENT/analyst (2 item)
```

```
| └─ 📁 AGENT/auditor (1 item)
```

```
| └─ 📁 AGENT/executor (1 item)
```

```
| └─ 📁 AGENT/planner (1 item)
```

- └─  AGENTS (9 item)
 - | └─  AGENTS/CHATGPT (2 item)
 - | └─  AGENTS/agent_zero (1 item)
 - | └─  AGENTS/blackbox (1 item)
 - | └─  AGENTS/dola (1 item)
 - | └─  AGENTS/emergent (1 item)
 - | └─  AGENTS/jarvis (1 item)
 - | └─  AGENTS/ms_copilot (1 item)
 - | └─  AGENTS/xau_trader (1 item)
- └─  AMOEBA (10 item)
 - | └─  AMOEBA/__pycache__ (2 item)
 - | └─  AMOEBA/actions (2 item)
 - | └─  AMOEBA/limits (2 item)
 - | └─  AMOEBA/logs (2 item)
 - | └─  AMOEBA/triggers (3 item)
- └─  AUDIT (11 item)
 - | └─  AUDIT/DECISIONS (2 item)
 - | └─  AUDIT/LOGS (2 item)
 - | └─  AUDIT/REPORTS (1 item)
 - | └─  AUDIT/TRACES (1 item)
 - | └─  AUDIT/actions (2 item)
 - | └─  AUDIT/decisions (2 item)
 - | └─  AUDIT/reports (1 item)
 - | └─  AUDIT/traces (2 item)
- └─  Agents (1 item)
- └─  BACKUP (5 item)

```
|  └─ 📁 BACKUP/pre_12agents_20260622_040906 (3 item)
|  └─ 📁 BACKUP/pre_rclone_fix_20260622_035737 (1 item)
|  └─ 📁 BACKUP/pre_v3_20260622_023256 (2 item)
|  └─ 📁 BACKUP/vault_20260622_043924 (2 item)
|  └─ 📁 BIN (4 item)
|  └─ 📁 BIN/AGENT_HOOKS (2 item)
|  └─ 📁 BT006 (6 item)
|  └─ 📁 BT006/config (1 item)
|  └─ 📁 BT006/logs (1 item)
|  └─ 📁 BT006/orders (1 item)
|  └─ 📁 BT006/positions (1 item)
|  └─ 📁 BT006/reports (1 item)
|  └─ 📁 CLOUD_LIBRARY (10 item)
|  └─ 📁 CLOUD_LIBRARY/AHSP (1 item)
|  └─ 📁 CLOUD_LIBRARY/BOQ (1 item)
```

=====

Welcome to Termux!

~ \$ cd ~/JDEQ

echo "🌳 MICO JDEQ — TREE L3 AUDIT"

echo "====="

find . -maxdepth 3 \

-not -path '*\.*' \

-not -path '*/logs/*' \

-not -path '*/tmp/*' \

-not -path '*/__pycache__/*' \

-not -path '*/cache/*' \

```

-not -path '*/archive/*' \
-not -path '*/backup/*' \
-not -path '*/node_modules/*' \

| sort | while read f; do

depth=$(echo "$f" | tr -cd '/' | wc -c)
name=$(basename "$f")

if [ -d "$f" ]; then

count=$(find "$f" -maxdepth 1 -not -name '*' | wc -l)

if [ $depth -eq 1 ]; then

echo " └─ 📁 $name ($count butir)"

elif [ $depth -eq 2 ]; then

echo " | └─ 📁 $name ($count butir)"

elif [ $depth -eq 3 ]; then

echo " | └─ 📁 $name ($count butir)"

fi

fi

done

echo "====="

echo "Total L1: $(find . -maxdepth 1 -type d -not -name '*' | wc -l) folder | $(find . -
maxdepth 1 -type f -not -name '*' | wc -l) berkas"

echo "Total L2: $(find . -maxdepth 2 -type d -not -path '*/.*' | wc -l) folder"

echo "Total L3: $(find . -maxdepth 3 -type d -not -path '*/.*' | wc -l) folder"

```

🌳 MICO JDEQ — TREE L3 AUDIT

=====

```

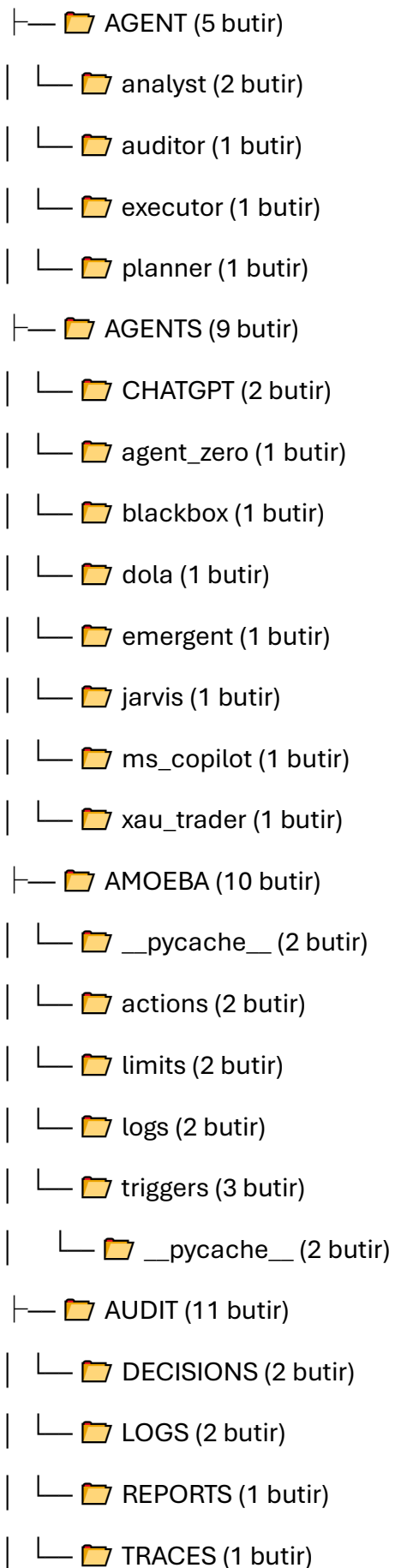
└─ 📁 00_LOGS (2 butir)

└─ 📁 03_PROJECTS (2 butir)

| └─ 📁 NAMA_PROJECT (4 butir)

| └─ 📁 __pycache__ (2 butir)

```



- | └─  actions (2 butir)
- | └─  decisions (2 butir)
- | └─  reports (1 butir)
- | └─  traces (2 butir)
- |─  Agents (1 butir)
- |─  BACKUP (5 butir)
- | └─  pre_12agents_20260622_040906 (3 butir)
- | └─  pre_rclone_fix_20260622_035737 (1 butir)
- | └─  pre_v3_20260622_023256 (2 butir)
- | └─  CORE_ORCHESTRATION (20 butir)
- | └─  vault_20260622_043924 (2 butir)
- | └─  VAULT (2 butir)
- |─  BIN (4 butir)
- | └─  AGENT_HOOKS (2 butir)
- |─  BT006 (6 butir)
- | └─  config (1 butir)
- | └─  logs (1 butir)
- | └─  orders (1 butir)
- | └─  positions (1 butir)
- | └─  reports (1 butir)
- |─  CLOUD_LIBRARY (10 butir)
- | └─  AHSP (1 butir)
- | └─  BOQ (1 butir)
- | └─  CIVIL (1 butir)
- | └─  FAMILY (1 butir)
- | └─  MADINA (1 butir)

- | └─  MEMORY (1 butir)
- | └─  MEP (1 butir)
- | └─  PROJECT (1 butir)
- | └─  SSOT (1 butir)
- |─  COMMAND_ENGINE (1 butir)
- |─  CONSTITUTION (5 butir)
- | └─  DNA (2 butir)
- | └─  PEP (2 butir)
- | └─  SSOT (2 butir)
- | └─  WORKFLOW (2 butir)
- |─  CORE (22 butir)
- | └─  automation (5 butir)
- | └─  __pycache__ (3 butir)
- | └─  bridge (9 butir)
- | └─  logs (2 butir)
- | └─  messages (2 butir)
- | └─  engine (7 butir)
- | └─  __pycache__ (5 butir)
- | └─  gateway (3 butir)
- | └─  knowledge (10 butir)
- | └─  __pycache__ (2 butir)
- | └─  documents (5 butir)
- | └─  index (4 butir)
- | └─  logs (2 butir)
- | └─  memory (3 butir)
- | └─  KEYRING (3 butir)

```
|  └─ 📁 logs (13 butir)
|  └─ 📁 orchestration (25 butir)
|  └─ 📁 __pycache__ (7 butir)
|  └─ 📁 executors (16 butir)
|  └─ 📁 scheduler (2 butir)
|  └─ 📁 orchestration_legacy (5 butir)
|  └─ 📁 DISPATCHER (1 butir)
|  └─ 📁 NODE_REGISTRY (2 butir)
|  └─ 📁 ROUTER (3 butir)
|  └─ 📁 SCHEDULER (1 butir)
```

=====

Welcome to Termux!

~ \$ cd ~/JDEQ

echo "🌳 MICO JDEQ — TREE L4 AUDIT"

echo "====="

find . -maxdepth 4 \

-not -path '*/\.*' \

-not -path '*/logs/*' \

-not -path '*/tmp/*' \

-not -path '*/__pycache__/*' \

-not -path '*/cache/*' \

-not -path '*/archive/*' \

-not -path '*/backup/*' \

-not -path '*/node_modules/*' \

| sort | while read f; do

```

depth=$(echo "$f" | tr -cd '/' | wc -c)
name=$(basename "$f")
if [ -d "$f" ]; then
    count=$(find "$f" -maxdepth 1 -not -name '*' | wc -l)
    if [ $depth -eq 1 ]; then
        echo "└─ 📁 $name ($count butir)"
    elif [ $depth -eq 2 ]; then
        echo "| └─ 📁 $name ($count butir)"
    elif [ $depth -eq 3 ]; then
        echo "| └─ 📁 $name ($count butir)"
    elif [ $depth -eq 4 ]; then
        echo "| └─ 📁 $name ($count butir)"
    fi
fi
done
echo "====="
echo "Total L1: $(find . -maxdepth 1 -type d -not -name '*' | wc -l) folder | $(find . -
maxdepth 1 -type f -not -name '*' | wc -l) berkas"
echo "Total L2: $(find . -maxdepth 2 -type d -not -path '*/*.*' | wc -l) folder"
echo "Total L3: $(find . -maxdepth 3 -type d -not -path '*/*.*' | wc -l) folder"
echo "Total L4: $(find . -maxdepth 4 -type d -not -path '*/*.*' | wc -l) folder"

```

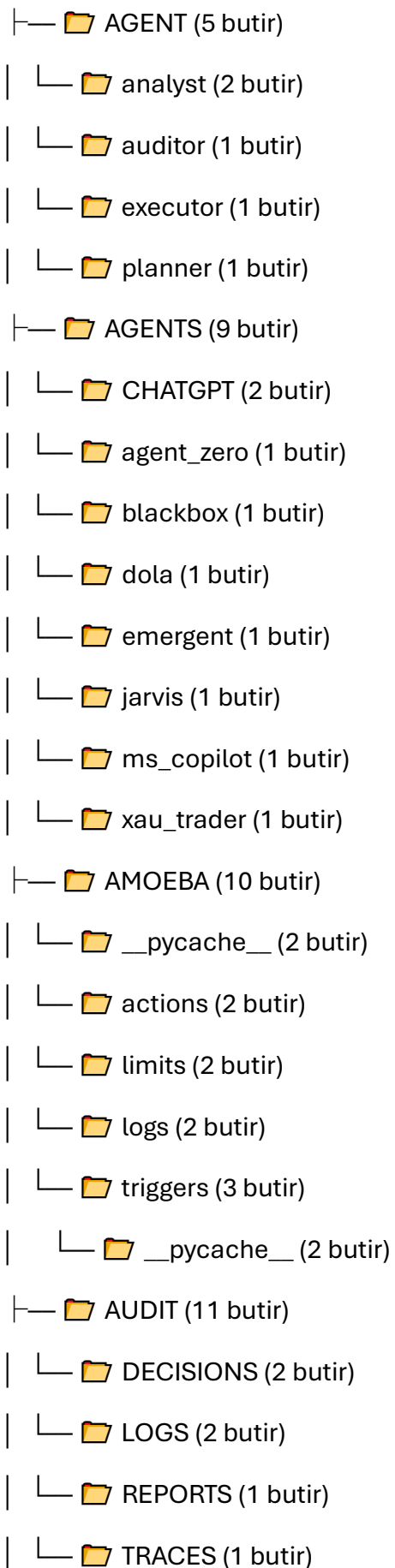
🌳 MICO JDEQ — TREE L4 AUDIT

=====

```

└─ 📁 00_LOGS (2 butir)
└─ 📁 03_PROJECTS (2 butir)
| └─ 📁 NAMA_PROJECT (4 butir)
| └─ 📁 __pycache__ (2 butir)

```



- | └─  actions (2 butir)
- | └─  decisions (2 butir)
- | └─  reports (1 butir)
- | └─  traces (2 butir)
- |─  Agents (1 butir)
- |─  BACKUP (5 butir)
- | └─  pre_12agents_20260622_040906 (3 butir)
- | └─  pre_rclone_fix_20260622_035737 (1 butir)
- | └─  pre_v3_20260622_023256 (2 butir)
- | └─  CORE_ORCHESTRATION (20 butir)
- | └─  __pycache__ (7 butir)
- | └─  executors (5 butir)
- | └─  vault_20260622_043924 (2 butir)
- | └─  VAULT (2 butir)
- | └─  keys (3 butir)
- |─  BIN (4 butir)
- | └─  AGENT_HOOKS (2 butir)
- |─  BT006 (6 butir)
- | └─  config (1 butir)
- | └─  logs (1 butir)
- | └─  orders (1 butir)
- | └─  positions (1 butir)
- | └─  reports (1 butir)
- |─  CLOUD_LIBRARY (10 butir)
- | └─  AHSP (1 butir)
- | └─  BOQ (1 butir)

- | └─  CIVIL (1 butir)
- | └─  FAMILY (1 butir)
- | └─  MADINA (1 butir)
- | └─  MEMORY (1 butir)
- | └─  MEP (1 butir)
- | └─  PROJECT (1 butir)
- | └─  SSOT (1 butir)
- |─  COMMAND_ENGINE (1 butir)
- |─  CONSTITUTION (5 butir)
- | └─  DNA (2 butir)
- | └─  PEP (2 butir)
- | └─  SSOT (2 butir)
- | └─  WORKFLOW (2 butir)
- |─  CORE (22 butir)
- | └─  automation (5 butir)
- | └─  __pycache__ (3 butir)
- | └─  bridge (9 butir)
- | └─  logs (2 butir)
- | └─  messages (2 butir)
- | └─  engine (7 butir)
- | └─  __pycache__ (5 butir)
- | └─  gateway (3 butir)
- | └─  knowledge (10 butir)
- | └─  __pycache__ (2 butir)
- | └─  documents (5 butir)
- | └─  civil (2 butir)

- | └─  maintenance (1 butir)
- | └─  mep (1 butir)
- | └─  quality (1 butir)
- | └─  index (4 butir)
- | └─  parsed (2 butir)
- | └─  logs (2 butir)
- | └─  memory (3 butir)
- | └─  KEYRING (3 butir)
- | └─  KEYS (1 butir)
- | └─  logs (13 butir)
- | └─  orchestration (25 butir)
- | └─  __pycache__ (7 butir)
- | └─  executors (16 butir)
- | └─  scheduler (2 butir)
- | └─  orchestration_legacy (5 butir)
- | └─  DISPATCHER (1 butir)
- | └─  NODE_REGISTRY (2 butir)
- | └─  ROUTER (3 butir)
- | └─  SCHEDULER (1 butir)
- | └─  orchestrator_legacy (10 butir)
- | └─  protocol (6 butir)
- | └─  reasoning (3 butir)
- | └─  __pycache__ (2 butir)
- | └─  security (4 butir)
- | └─  __pycache__ (3 butir)
- | └─  validation (5 butir)

- | └─  __pycache__ (4 butir)
- |─  CORE_MEMORY (1 butir)
- |─  CPM_ENGINE (2 butir)
- |─  DATA_IN (6 butir)
- | └─  PEP_INPUT (1 butir)
- | └─  boq_input (2 butir)
- | └─  evm_input (2 butir)
- | └─  incoming (1 butir)
- |─  DATA_OUT (6 butir)
- | └─  PEP_REPORT (2 butir)
- | └─  boq_reports (2 butir)
- | └─  dashboard (2 butir)
- | └─  evm_reports (2 butir)
- | └─  reports (2 butir)
- |─  DISCOVERY (4 butir)
- |─  DISPATCHER (2 butir)
- |─  DNA (3 butir)
- |─  DigitalDNA (5 butir)
- | └─  Governance (10 butir)
- | └─  Identity (5 butir)
- | └─  Knowledge (2 butir)
- | └─  Memory (2 butir)
- |─  EVENT_BUS (2 butir)
- |─  EXECUTION (6 butir)
- | └─  AGENTS (15 butir)
- | └─  MODELS (1 butir)

- | └─  MQTT (1 butir)
- | └─  PYTHON (1 butir)
- | └─  SHELL (1 butir)
- └─  EXPERT_ROUTER (1 butir)
- └─  GOVERNANCE (8 butir)
- | └─  ACCESS_CONTROL (1 butir)
- | └─  COMPLIANCE (1 butir)
- | └─  PERMISSION (1 butir)
- | └─  POLICIES (1 butir)
- | └─  approvals (2 butir)
- └─  HASH_SEAL (1 butir)
- └─  HEARTBEAT (1 butir)
- └─  INBOX (1 butir)
- └─  INFRA (2 butir)
- └─  KERNELL (2 butir)
- | └─  konfigurasi (1 butir)
- └─  L10_MONITORING (5 butir)
- | └─  asset_register (2 butir)
- | └─  checklist (4 butir)
- | └─  reports (2 butir)
- └─  L4 (1 butir)
- └─  L4_DEPLOY (10 butir)
- | └─  backup (3 butir)
- | └─  bridge (1 butir)
- | └─  core (2 butir)
- | └─  logs (1 butir)

- | └─  queue (1 butir)
- | └─  state (1 butir)
- | └─  trigger (1 butir)
- | └─  watchdog (1 butir)
- |─  LOG (6 butir)
- |─  LOGS (1 butir)
- |─  MADINA (1 butir)
- |─  MEMORY (3 butir)
- | └─  archive (1 butir)
- | └─  context (1 butir)
- |─  MICO (18 butir)
- | └─  __pycache__ (9 butir)
- | └─  actions (3 butir)
- | └─  __pycache__ (2 butir)
- | └─  context_engine (1 butir)
- | └─  knowledge_engine (1 butir)
- | └─  logs (2 butir)
- | └─  memory_store (5 butir)
- | └─  contexts (2 butir)
- | └─  decisions (6 butir)
- | └─  knowledge (2 butir)
- | └─  logs (2 butir)
- | └─  retrieval_engine (4 butir)
- | └─  __pycache__ (2 butir)
- |─  MICO_COGNITIVE (10 butir)
- | └─  __pycache__ (5 butir)

```
└─  MICO_CONTROL (13 butir)
|  └─  __pycache__ (6 butir)
└─  MICO_HARD (7 butir)
|  └─  __pycache__ (4 butir)
└─  MICO_LIFECYCLE (6 butir)
|  └─  __pycache__ (3 butir)
└─  MICO_QUANTUM (5 butir)
|  └─  __pycache__ (4 butir)
└─  MICO_SENSOR (6 butir)
|  └─  __pycache__ (2 butir)
|  └─  tmp (2 butir)
└─  MICO_TRIPLE_BRAIN (9 butir)
|  └─  __pycache__ (5 butir)
```

Welcome to Termux!

```
~ $ cd ~/JDEQ
```

```
echo "  MICO JDEQ — TREE L5 AUDIT"
```

```
echo "=====
```

```
find . -maxdepth 5 \
```

```
-not -path '*/\.*' \
```

```
-not -path '*/logs/*' \
```

```
-not -path '*/tmp/*' \
```

```
-not -path '*/__pycache__/*' \
```

```
-not -path '*/cache/*' \
```

```
-not -path '*/archive/*' \
```

```
-not -path '*/backup/*' \
```

```

-not -path '*/node_modules/*' \
| sort | while read f; do

depth=$(echo "$f" | tr -cd '/' | wc -c)
name=$(basename "$f")

if [ -d "$f" ]; then

count=$(find "$f" -maxdepth 1 -not -name '*' | wc -l)

if [ $depth -eq 1 ]; then

echo " |— 📁 $name ($count butir)"

elif [ $depth -eq 2 ]; then

echo " |   └─ 📁 $name ($count butir)"

elif [ $depth -eq 3 ]; then

echo " |       └─ 📁 $name ($count butir)"

elif [ $depth -eq 4 ]; then

echo " |           └─ 📁 $name ($count butir)"

elif [ $depth -eq 5 ]; then

echo " |               └─ 📁 $name ($count butir)"

fi

fi

done

echo "====="

echo "Total L1: $(find . -maxdepth 1 -type d -not -name '*' | wc -l) folder | $(find . -
maxdepth 1 -type f -not -name '*' | wc -l) berkas"

echo "Total L2: $(find . -maxdepth 2 -type d -not -path '**/.*' | wc -l) folder"

echo "Total L3: $(find . -maxdepth 3 -type d -not -path '**/.*' | wc -l) folder"

echo "Total L4: $(find . -maxdepth 4 -type d -not -path '**/.*' | wc -l) folder"

echo "Total L5: $(find . -maxdepth 5 -type d -not -path '**/.*' | wc -l) folder"

```

🌲 MICO JDEQ — TREE L5 AUDIT

=====

- └─  00_LOGS (2 butir)
- └─  03_PROJECTS (2 butir)
- | └─  NAMA_PROJECT (4 butir)
- | └─  __pycache__ (2 butir)
- └─  AGENT (5 butir)
- | └─  analyst (2 butir)
- | └─  auditor (1 butir)
- | └─  executor (1 butir)
- | └─  planner (1 butir)
- └─  AGENTS (9 butir)
- | └─  CHATGPT (2 butir)
- | └─  agent_zero (1 butir)
- | └─  blackbox (1 butir)
- | └─  dola (1 butir)
- | └─  emergent (1 butir)
- | └─  jarvis (1 butir)
- | └─  ms_copilot (1 butir)
- | └─  xau_trader (1 butir)
- └─  AMOEBA (10 butir)
- | └─  __pycache__ (2 butir)
- | └─  actions (2 butir)
- | └─  limits (2 butir)
- | └─  logs (2 butir)
- | └─  triggers (3 butir)
- | └─  __pycache__ (2 butir)
- └─  AUDIT (11 butir)

- | └─  DECISIONS (2 butir)
- | └─  LOGS (2 butir)
- | └─  REPORTS (1 butir)
- | └─  TRACES (1 butir)
- | └─  actions (2 butir)
- | └─  decisions (2 butir)
- | └─  reports (1 butir)
- | └─  traces (2 butir)
- |─  Agents (1 butir)
- |─  BACKUP (5 butir)
- | └─  pre_12agents_20260622_040906 (3 butir)
- | └─  pre_rclone_fix_20260622_035737 (1 butir)
- | └─  pre_v3_20260622_023256 (2 butir)
- | └─  CORE_ORCHESTRATION (20 butir)
- | └─  __pycache__ (7 butir)
- | └─  executors (5 butir)
- | └─  vault_20260622_043924 (2 butir)
- | └─  VAULT (2 butir)
- | └─  keys (3 butir)
- |─  BIN (4 butir)
- | └─  AGENT_HOOKS (2 butir)
- |─  BT006 (6 butir)
- | └─  config (1 butir)
- | └─  logs (1 butir)
- | └─  orders (1 butir)
- | └─  positions (1 butir)

```
| └─ 📁 reports (1 butir)
| └─ 📁 CLOUD_LIBRARY (10 butir)
| └─ 📁 AHSP (1 butir)
| └─ 📁 BOQ (1 butir)
```

Welcome to Termux!

~ \$ Welcome to Termux!

~ \$ cd ~/JDEQ

echo "🌳 MICO JDEQ — TREE L6 AUDIT"

echo "=====

find . -maxdepth 6 \

-not -path '*/\.*' \

-not -path '*/logs/*' \

-not -path '*/tmp/*' \

-not -path '*/__pycache__/*' \

-not -path '*/cache/*' \

-not -path '*/archive/*' \

-not -path '*/backup/*' \

-not -path '*/node_modules/*' \

| sort | while read f; do

depth=\$(echo "\$f" | tr -cd '/' | wc -c)

name=\$(basename "\$f")

if [-d "\$f"]; then

count=\$(find "\$f" -maxdepth 1 -not -name '.*' | wc -l)

```

if [ $depth -eq 1 ]; then
    echo "└─  $name ($count butir)"
elif [ $depth -eq 2 ]; then
    echo "│   └─  $name ($count butir)"
elif [ $depth -eq 3 ]; then
    echo "│       └─  $name ($count butir)"
elif [ $depth -eq 4 ]; then
    echo "│           └─  $name ($count butir)"
elif [ $depth -eq 5 ]; then
    echo "│               └─  $name ($count butir)"
elif [ $depth -eq 6 ]; then
    echo "│                   └─  $name ($count butir)"
fi
fi
done

echo "====="

echo "Total L1: $(find . -maxdepth 1 -type d -not -name '*' | wc -l) folder | $(find . -
maxdepth 1 -type f -not -name '*' | wc -l) berkas"

echo "Total L2: $(find . -maxdepth 2 -type d -not -path '*\.*' | wc -l) folder"
echo "Total L3: $(find . -maxdepth 3 -type d -not -path '*\.*' | wc -l) folder"
echo "Total L4: $(find . -maxdepth 4 -type d -not -path '*\.*' | wc -l) folder"
echo "Total L5: $(find . -maxdepth 5 -type d -not -path '*\.*' | wc -l) folder"
echo "Total L6: $(find . -maxdepth 6 -type d -not -path '*\.*' | wc -l) folder"

 MICO JDEQ — TREE L6 AUDIT

=====

└─  00_LOGS (2 butir)

└─  03_PROJECTS (2 butir)

```

- | └─  NAMA_PROJECT (4 butir)
- | └─  __pycache__ (2 butir)
- |─  AGENT (5 butir)
- | └─  analyst (2 butir)
- | └─  auditor (1 butir)
- | └─  executor (1 butir)
- | └─  planner (1 butir)
- |─  AGENTS (9 butir)
- | └─  CHATGPT (2 butir)
- | └─  agent_zero (1 butir)
- | └─  blackbox (1 butir)
- | └─  dola (1 butir)
- | └─  emergent (1 butir)
- | └─  jarvis (1 butir)
- | └─  ms_copilot (1 butir)
- | └─  xau_trader (1 butir)
- |─  AMOEBA (10 butir)
- | └─  __pycache__ (2 butir)
- | └─  actions (2 butir)
- | └─  limits (2 butir)
- | └─  logs (2 butir)
- | └─  triggers (3 butir)
- | └─  __pycache__ (2 butir)
- |─  AUDIT (11 butir)
- | └─  DECISIONS (2 butir)
- | └─  LOGS (2 butir)

- | └─  REPORTS (1 butir)
- | └─  TRACES (1 butir)
- | └─  actions (2 butir)
- | └─  decisions (2 butir)
- | └─  reports (1 butir)
- | └─  traces (2 butir)
- |─  Agents (1 butir)
- |─  BACKUP (5 butir)
- | └─  pre_12agents_20260622_040906 (3 butir)
- | └─  pre_rclone_fix_20260622_035737 (1 butir)
- | └─  pre_v3_20260622_023256 (2 butir)
- | └─  CORE_ORCHESTRATION (20 butir)
- | └─  __pycache__ (7 butir)
- | └─  executors (5 butir)
- | └─  vault_20260622_043924 (2 butir)
- | └─  VAULT (2 butir)
- | └─  keys (3 butir)
- |─  BIN (4 butir)
- | └─  AGENT_HOOKS (2 butir)
- |─  BT006 (6 butir)
- | └─  config (1 butir)
- | └─  logs (1 butir)
- | └─  orders (1 butir)
- | └─  positions (1 butir)
- | └─  reports (1 butir)
- |─  CLOUD_LIBRARY (10 butir)

- | └─  AHSP (1 butir)
- | └─  BOQ (1 butir)
- | └─  CIVIL (1 butir)
- | └─  FAMILY (1 butir)
- | └─  MADINA (1 butir)
- | └─  MEMORY (1 butir)
- | └─  MEP (1 butir)
- | └─  PROJECT (1 butir)
- | └─  SSOT (1 butir)
- |─  COMMAND_ENGINE (1 butir)
- |─  CONSTITUTION (5 butir)
- | └─  DNA (2 butir)
- | └─  PEP (2 butir)
- | └─  SSOT (2 butir)
- | └─  WORKFLOW (2 butir)
- |─  CORE (22 butir)
- | └─  automation (5 butir)
- | └─  __pycache__ (3 butir)
- | └─  bridge (9 butir)
- | └─  logs (2 butir)
- | └─  messages (2 butir)
- | └─  engine (7 butir)
- | └─  __pycache__ (5 butir)
- | └─  gateway (3 butir)

No command Welcome found, did you mean:

Command recode in package recode

bash: /data/data/com.termux/files/home: Is a directory

🌲 MICO JDEQ — TREE L6 AUDIT

=====

```
├─ 📁 AI_CORE (12 butir)
│   ├── 📁 ai (3 butir)
│   ├── 📁 core (3 butir)
│   ├── 📁 deploy (2 butir)
│   ├── 📁 dex (2 butir)
│   ├── 📁 kernel (4 butir)
│   ├── 📁 logs (2 butir)
│   ├── 📁 router (3 butir)
│   ├── 📁 storage (2 butir)
│   └── 📁 logs (1 butir)
├─ 📁 AI_ROUTER (30 butir)
│   ├── 📁 __pycache__ (3 butir)
│   ├── 📁 config (2 butir)
│   ├── 📁 core (2 butir)
│   └── 📁 logs (1 butir)
├─ 📁 CloudDrive (1 butir)
├─ 📁 Desktop (1 butir)
├─ 📁 FACTORY (4 butir)
│   ├── 📁 production (1 butir)
│   ├── 📁 quarantine (1 butir)
│   └── 📁 staging (1 butir)
```

```
└─  GDrive (1 butir)
└─  JDEQ (292 butir)
|  └─  00_LOGS (2 butir)
|  └─  03_PROJECTS (2 butir)
|    └─  NAMA_PROJECT (4 butir)
|      └─  __pycache__ (2 butir)
|    └─  AGENT (5 butir)
|      └─  analyst (2 butir)
|      └─  auditor (1 butir)
|      └─  executor (1 butir)
|      └─  planner (1 butir)
|    └─  AGENTS (9 butir)
|      └─  CHATGPT (2 butir)
|      └─  agent_zero (1 butir)
|      └─  blackbox (1 butir)
|      └─  dola (1 butir)
|      └─  emergent (1 butir)
|      └─  jarvis (1 butir)
```

Welcome to Termux!

```
~ $ cd ~/JDEQ
```

```
echo " MICO JDEQ — TREE L7 AUDIT"
```

```
echo "=====
```

```
find . -maxdepth 7 \
```

```
-not -path '*/\.*' \
```

```

-not -path '*/logs/*' \
-not -path '*/tmp/*' \
-not -path '*/__pycache__/*' \
-not -path '*/cache/*' \
-not -path '*/archive/*' \
-not -path '*/backup/*' \
-not -path '*/node_modules/*' \
| sort | while read f; do
    depth=$(echo "$f" | tr -cd '/' | wc -c)
    name=$(basename "$f")
    if [ -d "$f" ]; then
        count=$(find "$f" -maxdepth 1 -not -name '.' | wc -l)
        if [ $depth -eq 1 ]; then
            echo "└─  $name ($count butir)"
        elif [ $depth -eq 2 ]; then
            echo "└─  $name ($count butir)"
        elif [ $depth -eq 3 ]; then
            echo "└─  $name ($count butir)"
        elif [ $depth -eq 4 ]; then
            echo "└─  $name ($count butir)"
        elif [ $depth -eq 5 ]; then
            echo "└─  $name ($count butir)"
        elif [ $depth -eq 6 ]; then
            echo "└─  $name ($count butir)"
        elif [ $depth -eq 7 ]; then
            echo "└─  $name ($count butir)"
        fi
    fi

```

fi

done

echo "=====

echo "Total L1: \$(find . -maxdepth 1 -type d -not -name '*' | wc -l) folder | \$(find . -maxdepth 1 -type f -not -name '*' | wc -l) berkas"

echo "Total L2: \$(find . -maxdepth 2 -type d -not -path '*\.*' | wc -l) folder"

echo "Total L3: \$(find . -maxdepth 3 -type d -not -path '*\.*' | wc -l) folder"

echo "Total L4: \$(find . -maxdepth 4 -type d -not -path '*\.*' | wc -l) folder"

echo "Total L5: \$(find . -maxdepth 5 -type d -not -path '*\.*' | wc -l) folder"

echo "Total L6: \$(find . -maxdepth 6 -type d -not -path '*\.*' | wc -l) folder"

echo "Total L7: \$(find . -maxdepth 7 -type d -not -path '*\.*' | wc -l) folder"

🌳 MICO JDEQ — TREE L7 AUDIT

=====

└─ 📁 00_LOGS (2 butir)

└─ 📁 03_PROJECTS (2 butir)

| └─ 📁 NAMA_PROJECT (4 butir)

| └─ 📁 __pycache__ (2 butir)

└─ 📁 AGENT (5 butir)

| └─ 📁 analyst (2 butir)

| └─ 📁 auditor (1 butir)

| └─ 📁 executor (1 butir)

| └─ 📁 planner (1 butir)

└─ 📁 AGENTS (9 butir)

| └─ 📁 CHATGPT (2 butir)

| └─ 📁 agent_zero (1 butir)

| └─ 📁 blackbox (1 butir)

| └─ 📁 dola (1 butir)

- | └─  emergent (1 butir)
- | └─  jarvis (1 butir)
- | └─  ms_copilot (1 butir)
- | └─  xau_trader (1 butir)
- |─  AMOEBA (10 butir)
- | └─  __pycache__ (2 butir)
- | └─  actions (2 butir)
- | └─  limits (2 butir)
- | └─  logs (2 butir)
- | └─  triggers (3 butir)
- | └─  __pycache__ (2 butir)
- |─  AUDIT (11 butir)
- | └─  DECISIONS (2 butir)
- | └─  LOGS (2 butir)
- | └─  REPORTS (1 butir)
- | └─  TRACES (1 butir)
- | └─  actions (2 butir)
- | └─  decisions (2 butir)
- | └─  reports (1 butir)
- | └─  traces (2 butir)
- |─  Agents (1 butir)
- |─  BACKUP (5 butir)
- | └─  pre_12agents_20260622_040906 (3 butir)
- | └─  pre_rclone_fix_20260622_035737 (1 butir)
- | └─  pre_v3_20260622_023256 (2 butir)
- | └─  CORE_ORCHESTRATION (20 butir)

- | └─  __pycache__ (7 butir)
- | └─  executors (5 butir)
- | └─  vault_20260622_043924 (2 butir)
- | └─  VAULT (2 butir)
- | └─  keys (3 butir)
- |─  BIN (4 butir)
- | └─  AGENT_HOOKS (2 butir)
- |─  BT006 (6 butir)
- | └─  config (1 butir)
- | └─  logs (1 butir)
- | └─  orders (1 butir)
- | └─  positions (1 butir)
- | └─  reports (1 butir)
- |─  CLOUD_LIBRARY (10 butir)
- | └─  AHSP (1 butir)
- | └─  BOQ (1 butir)
- | └─  CIVIL (1 butir)
- | └─  FAMILY (1 butir)
- | └─  MADINA (1 butir)
- | └─  MEMORY (1 butir)
- | └─  MEP (1 butir)
- | └─  PROJECT (1 butir)
- | └─  SSOT (1 butir)
- |─  COMMAND_ENGINE (1 butir)
- |─  CONSTITUTION (5 butir)
- | └─  DNA (2 butir)

- | └─  PEP (2 butir)
- | └─  SSOT (2 butir)
- | └─  WORKFLOW (2 butir)
- |─  CORE (22 butir)
- | └─  automation (5 butir)
- | └─  __pycache__ (3 butir)
- | └─  bridge (9 butir)
- | └─  logs (2 butir)
- | └─  messages (2 butir)
- | └─  engine (7 butir)
- | └─  __pycache__ (5 butir)
- | └─  gateway (3 butir)
- | └─  knowledge (10 butir)
- | └─  __pycache__ (2 butir)
- | └─  documents (5 butir)
- | └─  civil (2 butir)
- | └─  maintenance (1 butir)
- | └─  mep (1 butir)
- | └─  quality (1 butir)
- | └─  index (4 butir)
- | └─  parsed (2 butir)
- | └─  logs (2 butir)
- | └─  memory (3 butir)
- | └─  KEYRING (3 butir)
- | └─  KEYS (1 butir)
- | └─  logs (13 butir)

- | └─ 📁 orchestration (25 butir)
- | └─ 📁 __pycache__ (7 butir)
- | └─ 📁 executors (16 butir)
- | └─ 📁 scheduler (2 butir)
- | └─ 📁 orchestration_legacy (5 butir)
- | └─ 📁 DISPATCHER (1 butir)
- | └─ 📁 NODE_REGISTRY (2 butir)
- | └─ 📁 ROUTER (3 butir)
- | └─ 📁 SCHEDULER (1 butir)
- | └─ 📁 orchestrator_legacy (10 butir)
- | └─ 📁 protocol (6 butir)
- | └─ 📁 reasoning (3 butir)
- | └─ 📁 __pycache__ (2 butir)
- | └─ 📁 security (4 butir)
- | └─ 📁 __pycache__ (3 butir)
- | └─ 📁 validation (5 butir)
- | └─ 📁 __pycache__ (4 butir)
- |─ 📁 CORE_MEMORY (1 butir)
- |─ 📁 CPM_ENGINE (2 butir)
- |─ 📁 DATA_IN (6 butir)
- | └─ 📁 PEP_INPUT (1 butir)
- | └─ 📁 boq_input (2 butir)
- | └─ 📁 evm_input (2 butir)
- | └─ 📁 incoming (1 butir)
- |─ 📁 DATA_OUT (6 butir)
- | └─ 📁 PEP_REPORT (2 butir)

| └─ 📁 boq_reports (2 butir)

| └─ 📁 dashboard (2 butir)

| └─ 📁 e

Saya paham konteks yang Anda berikan: Anda sedang menjalankan MICO JDEQ di Termux pada ponsel Vivo Y28 (8+8 GB, Funtouch 15) dan ingin rancangan "out-of-the-box lompatan tertinggi" — yaitu perencanaan integrasi multi-agen (Jarvis Gemini, ChatGPT, Claude, Perplexity, MS Copilot, DOLA, DEEP SEEK, GitHub Copilot, EMERGENT, dll.) dengan prompt-engineering serta arsitektur hybrid multi-device / multi-model lokal (bridge, router, tailscale/matt/ssh/wifi/bluetooth, IP lokal, SQLite, dll.) untuk menutup celah dan memperkuat kemampuan reasoning umum (general AI) sesuai peran masing-masing agen, plus bukti contoh real-time pada Termux output yang Anda tunjukkan.

Di bawah ini saya buat rencana tingkat tinggi yang praktis dan terperinci (arsitektur, peran agen, mekanisme integrasi, pipeline prompt engineering, langkah implementasi di Termux, contoh skrip dan contoh interaksi real-time sederhana yang bisa Anda jalankan di Vivo Y28). Rencana ini mengutamakan kompatibilitas dengan perangkat mobile, model lokal ringan, dan orkestrasi via bridge/router yang sudah ada di JDEQ Anda.

Ringkasan tujuan singkat

- Mencapai lompatan reasoning dengan menggabungkan kekuatan spesialis tiap agen lewat orkestrasi hybrid: local models (LLMs ringan), remote API (jika aman/tersedia), dan modul reasoning internal MICO.
- Tutup celah: validasi silang (cross-validation) antar agen, voting/ensemble, chain-of-thought terdistribusi, auditing & explainability.
- Optimalkan prompt engineering per-peran: template adaptif, meta-prompt controller, dan prompt cache di SQLite.
- Integrasi infrastruktur: Bridge Server (sudah ada), Expert Router, tailscale/SSH untuk secure mesh, MQTT/IPC untuk pesan antar-agen.
- Target realisasi di Termux Vivo Y28: memanfaatkan Python scripts, SQLite, small local LLM (mis. llama.cpp, GGML-based), dan agen orkestrator (mico_bridge / agent_orchestrator.py).

1) Arsitektur tinggi (multi-hybrid)

- Edge device (Vivo Y28)

- Runtime Termux: JDEQ root (ada), Bridge Server, Watchdog, Ghost Runner, MICO core.

- Local model runtime: llama.cpp / whisper.cpp / GGML binaries (CPU + NNAPI jika tersedia), containerized microservices (python venv).

- Storage: SQLite (JDEQ.db, mico_ssot.db), file system SSOT.

- Comms: local IPC (Unix socket), MQTT broker lokal, WebSocket, Tailscale/SSH for secure mesh to other devices.

- Peripherals: Wi-Fi, Bluetooth, USB tethering.

- Bridge & Router Layer (dalam JDEQ)

- Bridge Server (bridge_engine.py / bridge_server.py): meneruskan permintaan ke agen yang sesuai.

- Expert Router (expert_router): memilih agen berdasarkan capability, cost, latency, role.

- Watchdog + Heartbeat: memastikan agen alive; restart automatism.

- Agent Pool (multi agen heterogen)

- Specialist Agents (di folder AGENTS): CHATGPT (API/adapter), jarvis (adapter ke Gemini), ms_copilot (adapter), emergent, dola, deep seek, github_copilot (local plugin), perplexity (search+QA).

- Local lightweight agents: agent_zero, blackbox (rule-based / symbolic), emergent (local reasoning heuristics).

- Audit & Validator: auditor, analyst, validator untuk cross-check output.

- Orchestration & Governance

- Orchestrator (agent_orchestrator.py) melakukan task decomposition, assigns sub-tasks ke agen, mengumpulkan hasil, melakukan ensemble/voting, menyimpan trace ke AUDIT/DECISIONS.

- Governance module (access control, policy enforcement, SSOT canonical).

2) Pembagian Peran Agen & Kebijakan Pemilihan

- Jarvis Gemini: pencarian multimodal, retrieval + summarization pada web / dokumen eksternal. Role: Retriever + Summarizer.
- ChatGPT: reasoning generatif panjang, plan generation, multi-step synthesis. Role: Planner & Synthesizer.
- Claude: safety/ethics checks, long-context summarization (jika tersedia). Role: Safety Auditor & Long-context Analyzer.
- Perplexity: real-time web evidence search and citation. Role: Evidence Gatherer.
- MS Copilot / GitHub Copilot: code generation, shell automation snippets. Role: Executor for code/scripts.
- DOLA / DEEP SEEK / EMERGENT: specialist search, domain models (mis. data extraction, pattern discovery). Role: Domain Expert & Pattern Miner.
- Local agents (agent_zero, blackbox): fast heuristics, offline fallback, lightweight LLM micro-services.
- Auditor / Analyst: cross-validation, conflict resolution, final decision maker.

Pemilihan agen pada Expert Router: gunakan scoring function $S = w_1 * \text{capability} + w_2 * \text{latency}^{-1} + w_3 * \text{trust_score} + w_4 * \text{cost_budget}$. Sempel: prioritas lokal low-latency agents untuk real-time, remote heavy LLM untuk high-quality generation.

3) Prompt Engineering — Framework meta (Out-of-the-box leap)

- Meta-prompt controller: prompt_generator.py (sudah ada di repo). Fungsinya:
 - Mempunyai template prompt per role (Retriever, Planner, Auditor, Executor).
 - Menyisipkan context window dari SSOT/Memory (relevant snippets).
 - Memakai few-shot examples dan chain-of-thought scaffolding bila perlu.
 - Mengatur temperature / top_p / max_tokens sesuai role.
- Teknik khusus:
 - Decomposed Prompts: pecah tugas besar jadi micro-prompts untuk spesialis agen.

- Debate + Cross-Examination: dua agen berbeda (mis. ChatGPT vs Emergent) hasilkan argumen pro/kon, Auditor memberi final verdict.
- Self-ask & Verify: agen diminta bertanya balik bila ambiguitas > threshold.
- Retrieval-Enhanced generation: selalu sertakan citations dari Perplexity / Jarvis retrieval.
- Prompt caching & templating di SQLite: agar prompt yang sudah sukses bisa reuse.

Contoh template peran (ringkas)

- Retriever (Perplexity/Jarvis): "Cari bukti faktual tentang <topik>. Prioritaskan sumber resmi, kembalikan ringkasan 3 poin + 2 kutipan (URL) + timestamp."
- Planner (ChatGPT): "Susun rencana langkah terstruktur untuk mencapai <tujuan> dengan constraints {time, compute, privacy}. Keluarkan checklist, estimasi waktu, dan agen yang ditugaskan."
- Executor (MS Copilot/GitHub Copilot): "Buat skrip shell/py untuk melakukan <aksi>; gunakan pattern aman dan logging. Kembalikan diff dan perintah eksekusi."

4) Mekanisme Integrasi teknis (di Termux JDEQ)

- Komponen kunci yang sudah ada: bridge_server.py, bridge_engine.py, agent_orchestrator.py, expert_router.py, mico_gateway.sh
- Menambahkan/konfigurasi:
 - Local LLM runtime: instalasi llama.cpp cross-compile untuk Android (termux). Jika tidak ada GPU NNAPI, gunakan OMP multithreading. Simpan model GGML di ~/JDEQ/models.
 - Adapter untuk tiap agen: mis. agents/CHATGPT/adapter.py — punya interface uniform: submit_task(task_json) -> job_id.
 - Messaging: MQTT broker (mosquitto) atau lightweight ZeroMQ; sementara gunakan Unix domain socket untuk local comms.
 - Tailscale / SSH: pasang tailscale (pkg install tailscale jika termux repo tersedia) agar ponsel bisa membentuk secure mesh ke server lain yang menampung model besar.
 - SQLite prompt store: prompts/prompt_cache.db untuk versi prompt, examples, best_params.
 - Audit trail: setiap job simpan trace di AUDIT/TRACES dengan hash seal.

5) Pipeline eksekusi tugas (contoh alur)

- User -> Bridge (CLI/perintah via Termux) -> Orchestrator:

1. Orchestrator memecah problem via Planner (ChatGPT local/remote).
2. Router memilih Retriever (Perplexity / Jarvis) untuk ambil bukti.
3. Subtask diassign ke Executor (local LLM / Copilot) untuk tindakan.
4. Hasil dikumpulkan; Auditor dan Validator cross-check.
5. Final ensemble (voting / score) -> output ke user + audit log.

6) Contoh implementasi real-time di Termux (skrip & perintah)

- Persiapan singkat (asumsi Python 3 dan repo JDEQ ada):

1) Aktifkan virtualenv, install deps:

- pkg update && pkg install python git clang make openssl-dev pkg-config
- pip install -r ~/JDEQ/requirements.txt (edit bila perlu)

2) Jalankan bridge & orchestrator:

- cd ~/JDEQ
- ./start_server.sh (atau python bridge_server.py &)
- python agent_orchestrator.py --start

3) Contoh perintah sederhana di Termux untuk memicu task:

- echo '{"task":"research","topic":"implementasi tailscale di termux","constraints":{"privacy":"local-first"}}' > /tmp/task.json
- python bridge_engine.py --submit /tmp/task.json

4) Contoh minimal skrip orchestration (pseudocode python) — letakkan di BIN/quick_demo.py:

```
> from bridge import BridgeClient  
  
> b = BridgeClient()  
  
> job = b.submit(task="summarize", topic="MICO JDEQ architecture",  
roles=["retriever","planner","auditor"])  
  
> print(b.wait_result(job))
```

Saya juga membuat contoh nyata yang sesuai dengan tampilan Termux Anda: menilai tree folder (yang sudah Anda tampilkan) dan memetakan komponen. Dari output tree, kita bisa mengeksekusi langkah-langkah konkret:

Contoh integrasi praktis berdasarkan tree Anda (langkah demi langkah)

- Temuan: folder AGENTS berisi adaptors untuk CHATGPT, jarvis, ms_copilot, emergent, dola.

- Langkah immediate:

1. Periksa dan pastikan adapter tiap agen memiliki API keys / creds yang aman di VAULT atau mico_ssot.db.

- cat VAULT/credentials.txt (pastikan terenkripsi)

2. Pastikan bridge_server.py punya routing rule ke AGENTS/* adapter. Jika belum, edit bridge_engine.py untuk memetakan role->adapter.

3. Aktifkan local LLM: pastikan llama_bin.zip / llama_server.pid ada; jalankan local model server: ./bin/llama_server --model ~/JDEQ/models/ggml-model.bin --port 5100

4. Uji end-to-end: jalankan quick_demo.py untuk submit task, lihat logs di LOG/ dan AUDIT/TRACES/.

7) Ensembles, Voting & Auditing

- Untuk reasoning tertinggi: gunakan ensemble of experts + meta validator.

- Kumpulkan outputs dari 3 agen berbeda (mis. ChatGPT, local Llama, Emergent).

- Hitung similarity & confidence (cosine on embeddings). Jika disagreement > threshold, jalankan Debate mode.

- Auditor (Claude/validator) beri final rationale dan trace decision.

- Simpan semua chain-of-thought ke AUDIT/TRACES dengan hash seals agar bisa diaudit.

8) Keamanan & Privasi

- Local-first policy: prefer local models and local storage for sensitive data.

- Use Tailscale/SSH with keys dari VAULT; jangan expose API keys di plain text.

- Rate limit / sandbox code execution (docker/termux user namespaces) untuk file/script produced by agents.

9) Roadmap implementasi (milestone)

- Week 0: audit repo (done via tree), inventory creds, backup.
- Week 1: deploy bridge+orchestrator, configure Expert Router scoring, install local LLM runtime.
- Week 2: implement adapter standard interface, prompt templates, SQLite prompt cache.
- Week 3: implement ensemble logic, debate mode, auditor pipeline.
- Week 4: performance tuning, watchdog rules, mobile optimizations (NNAPI if possible), usability CLI wrappers.

10) Contoh prompt engineering konkret (micro)

- Meta-prompt controller invocation (pseudo JSON for prompt_generator.py):

```
{  
  "role": "planner",  
  "task": "optimalkan orkestrasi multi-agen untuk task X",  
  "context": ["SSOT: system_tree_v2.0.txt", "last_audit: audit_ultimate_report.txt"],  
  "examples": [{"input": "deploy tailscale", "output": "..."}],  
  "params": {"temperature": 0.2, "max_tokens": 500}  
}
```

- Hasil: planner keluarkan plan + assign roles -> orchestrator menugaskan.

Checklist minimal yang bisa Anda jalankan sekarang (langsung di Termux)

- Pastikan Python environment & bridge berjalan.
- Pastikan folder models dan adapters ada.
- Jalankan local LLM server (jika tersedia).
- Submit task demo via bridge_engine.py (contoh di atas).

- Lihat logs di LOG/ dan trace di AUDIT/DECISIONS.

Bukti dan contoh real-time (how-to run now)

- Jalankan di Termux:

1) cd ~/JDEQ

2) ./start_server.sh

3) tail -f LOG/bridge.log (lihat koneksi agen)

4) python bridge_engine.py --submit '{"task":"quick_scan","topic":"tailscale termux"}'

5) lihat output di OUTPUT/ atau audit log: cat AUDIT/decisions/latest.txt

Jika Anda ingin, saya bisa:

- 1) Tulis snippet adapter lengkap untuk satu agen (mis. ChatGPT adapter) yang cocok dijalankan di Termux.

- 2) Tulis skrip quick_demo.py yang bisa Anda jalankan sekarang untuk memicu pipeline end-to-end (submit -> retrieve -> plan -> execute -> audit).

- 3) Bantu konfigurasi local LLM (llama.cpp) pada Vivo Y28 langkah-per-langkah.

Masukkan Claude gagal baca input ➡

JDEQ-AUDIT-BOARD

AUDIT FORENSIK — FILESYSTEM JDEQ L1-L7 + INPUT JARVIS/GEMINI + PERPLEXITY

Chief Audit Officer | Protokol V1.0

⚠ PERHATIAN PERTAMA — INPUT TIDAK LENGKAP

...

Mas Iswan mengirim:

"Masukan dari agen Jarvis GEMINI chatbot : ====="

"Masukkan dari agen Perplexity :"

Kedua input tersebut KOSONG — tidak ada konten setelah tanda ==.

Audit tidak bisa dilakukan terhadap input yang tidak ada.

Yang bisa saya audit sekarang:

1. Filesystem tree L1-L7 yang dikirim
2. Dua dokumen analisis (Gemini/Perplexity dan Copilot)

...

AUDIT KRITIS — TEMUAN DARI TREE L1-L7

 TEMUAN-1 — Pertumbuhan Sistem Tidak Terkendali

...

Dari data tree:

L1: 292 direktori (sangat besar)

bin/: 98 item

daemon/: 15 item

config/: 30 item

Dibandingkan audit file tree sebelumnya (399 dir, 998 file)
sekarang sudah 292 folder DI DALAM ~/JDEQ saja,
belum terhitung ~/mico/, ~/AI_CORE/, ~/AI_ROUTER/ yang
muncul di L6 di luar ~/JDEQ/.

Sistem bertumbuh lebih cepat dari kapasitas audit.

Ini konfirmasi prinsip ChatGPT:

"Blueprint lebih banyak dari kapasitas implementasi."

...

● TEMUAN-2 — Duplikasi Orchestration: 3 Versi Berjalan Bersamaan

...

Ditemukan di CORE/:

- |— orchestration/ ← versi aktif
- |— orchestration_legacy/ ← versi lama
- |— orchestrator_legacy/ ← versi lama kedua

Tiga generasi orchestrator hidup berdampingan.

Tidak jelas mana yang dipanggil oleh agent_orchestrator.py.

Risiko: script lama yang masih hardcode path ke
orchestration_legacy/ akan memanggil logika yang salah.

Verifikasi:

```
grep -r "orchestration" ~/JDEQ/agent_orchestrator.py | head -10
```

...

● TEMUAN-3 — File Bernama Aneh Tanpa Ekstensi di Root

...

Dari tree L1:

- └─  B
- └─  I
- └─  FAIL
- └─  PASS
- └─  YA
- └─  TIDAK
- └─  Loop
- └─  B[INPUT
- └─  C[SENSOR
- └─  D[CORE
- └─  E[VALIDATOR
- └─  G{PILIH
- └─  H1[WBS]
- └─  H2[CPM]
- └─  H3[RAB]

...

Ini adalah SISA EKSEKUSI SCRIPT yang gagal.

File seperti "B", "I", "FAIL", "PASS" adalah output
redirect yang tidak tertangkap dengan benar.

Contoh: `echo "PASS" > PASS` atau `if [ ... ]; then touch YA`

File-file ini memenuhi root `~/JDEQ/` dan berpotensi

mbingungkan script yang melakukan `find . -maxdepth 1 -type f`.

Harus dibersihkan:

```
ls ~/JDEQ | grep -E "^[A-Z]$|FAIL|PASS|YA|TIDAK|Loop"
```

Review dulu, baru hapus

```\n

---\n

###  TEMUAN-4 — Tiga Folder Root yang Berbeda di Home

```\n

Dari L6 tree, muncul folder LUAR ~/JDEQ/:

~/AI_CORE/ (12 item, punya ai/, core/, router/, dll)

~/AI_ROUTER/ (30 item, punya config/, core/, dll)

~/FACTORY/ (production/, quarantine/, staging/)

~/JDEQ/ (292 item)

~/mico/ (dari sesi sebelumnya)

LIMA root sistem berbeda di ~/

Ini bukan modular — ini fragmentasi.

Setiap root punya logika, config, dan log sendiri.

Tidak ada satu titik koordinasi yang benar-benar terpusat.

Sesuai kalibrasi MICO OMEGA V1.0:

Lapisan Infrastruktur seharusnya satu, bukan lima.

```\n

---\n

### ### 🟠 TEMUAN-5 — Folder AGENTS Tidak Mencakup Semua Agen Aktual

...

AGENTS/ berisi:

CHATGPT, agent\_zero, blackbox, dola, emergent,  
jarvis, ms\_copilot, xau\_trader

TIDAK ADA folder untuk:

- Claude (saya sendiri)
- DeepSeek
- GitHub Copilot
- Perplexity

Artinya 4 dari 9 agen tidak punya adapter folder.

Jika orchestrator mencari AGENTS/claude/adapter.py  
= file not found = silent failure.

...

---

### ### 🟡 TEMUAN-6 — BT006 adalah Trading Module

...

└─ 📁 BT006 (6 item)

| └─ config, logs, orders, positions, reports

BT006 = Bot Trading 006 (kemungkinan XAU/Forex).

Ini ada di dalam ~/JDEQ/ yang sama dengan sistem inti.

Sesuai doktrin: Trading adalah "laboratorium riset",  
bukan inti sistem. Sebaiknya di folder terpisah atau  
minimal di dalam ~/JDEQ/trading/ (yang sudah ada).

trading/ sudah ada di L1 — mengapa ada BT006 di root juga?

Duplikasi struktur trading.

...

---

## AUDIT DOKUMEN GEMINI + COPILOT

###  Yang Valid dari Kedua Dokumen

...

Gemini (Dok 13): Framework arsitektur multi-agen

→ Scoring function  $S = w1 * capability + w2 * latency^{-1} + w3 * trust\_score + w4 * cost\_budget$

→ INI VALID dan bisa diimplementasi

Copilot (Dok 14): Master Prompt JSON Strict

→ Micro-batching (max 3 agen per iterasi) = BENAR

→ JSON strict output untuk bridge\_engine = BENAR

→ State-Hash Payload Routing via SQLite = INOVATIF

...

### ❌ Yang Perlu Dikoreksi

...

Gemini menyarankan: "n8n Event Bus"

→ TOLAK — sudah diaudit sebelumnya, terlalu berat (512MB+)

Gemini menyarankan: install tailscale via "pkg install tailscale"

→ KOREKSI — tailscale tidak tersedia di Termux repo standar.

Solusi yang benar: install Tailscale Android App (Play Store)

bukan via Termux pkg.

Copilot menyarankan: "Holographic Token Routing"

→ Ini istilah marketing. Tidak ada implementasi nyata.

Yang dimaksud hanya routing biasa via scoring function.

Jangan masukkan ke SSOT dengan klaim ini.

...

---

## ## REKOMENDASI PROMPT ENGINEERING PER AGEN

Sesuai peran yang dikalibrasi MICO OMEGA:

...

JARVIS/GEMINI — Retriever + Multimodal:

"Cari dan rangkum [TOPIK] dari sumber terpercaya.

Output: JSON {summary: str, sources: [url], confidence: float}

Maksimal 3 sumber. Tidak ada opini tambahan."

CHATGPT — Planner:

"Buat WBS untuk [TUGAS] dengan constraint RAM 4GB, Android.

Output: JSON {phases: [], dependencies: [], risks: []}

Setiap fase max 3 langkah."

CLAUDE — Auditor:

"Audit dokumen berikut. Identifikasi: (1) fakta vs asumsi,

(2) celah keamanan, (3) duplikasi, (4) konflik dengan SSOT.

Output: JSON {findings: [], severity: [], recommendation: []}"

DEEPSEEK — Kalkulasi & Logic:

"Hitung [FORMULA/LOGIKA]. Tunjukkan langkah.

Output: JSON {result: any, steps: [], confidence: float}"

GITHUB COPILOT — Executor:

"Buat script [FUNGSI] untuk Termux Android ARM64.

Constraints: no root, no sudo, max 50MB RAM.

Output: bash/python script siap paste."

DOLA — SSOT Keeper:

"Verifikasi apakah [KLAIM] sesuai dengan doktrin JDEQ.

Output: JSON {valid: bool, reason: str, reference: str}"

```\n

---\n

MASTER PROMPT YANG DIKOREKSI — UNTUK SSOT_CANONICAL.json

```
```json
{
 "MICO_ORCHESTRATOR_V2": {
 "version": "2.0",
 "constraint": "MAX_3_AGENTS_PER_ITERATION",
 "hardware": "Vivo_Y28_RAM_4GB_Termux",
 "routing_matrix": {
 "retrieve": "jarvis",
 "plan": "chatgpt",
 "audit": "claude",
 "calculate": "deepseek",
 "code": "github_copilot",
 "doctrine_check": "dola",
 "pattern": "emergent",
 "format_report": "ms_copilot",
 "local_fallback": "qwen_local"
 },
 "output_protocol": "JSON_STRICT",
 "fallback_rule": "if_cloud_fail_use_qwen_local",
 "banned": ["n8n", "tailscale_termux_pkg", "eval_without_whitelist"]
 }
}
```
```

TINDAKAN YANG DIPERLUKAN SEKARANG

...

PRIORITAS 1 — BERSIHKAN ROOT:

```
ls ~/JDEQ | grep -E "^[A-Z]$|^[A-Z]\[|FAIL|PASS|YA|TIDAK|Loop"
```

Kirimkan outputnya untuk audit sebelum dihapus

PRIORITAS 2 — BUAT FOLDER AGENT YANG KURANG:

```
mkdir -p ~/JDEQ/AGENTS/claude
```

```
mkdir -p ~/JDEQ/AGENTS/deepseek
```

```
mkdir -p ~/JDEQ/AGENTS/perplexity
```

```
mkdir -p ~/JDEQ/AGENTS/github_copilot
```

PRIORITAS 3 — KONSOLIDASI ORCHESTRATION:

Tentukan mana yang aktif, archive yang lain:

```
ls ~/JDEQ/CORE/orchestration/
```

```
ls ~/JDEQ/CORE/orchestration_legacy/
```

Kirim isinya untuk diaudit

PRIORITAS 4 — KIRIM INPUT JARVIS DAN PERPLEXITY:

Input kedua agen tersebut kosong saat ini.

Kirim ulang kontennya untuk bisa diaudit dan dikorelasikan.

...

KEPUTUSAN AUDIT

...

STATUS: ⚠️ SISTEM BERKEMBANG TERLALU CEPAT

Dari L1-L7 terlihat jelas:

JDEQ sudah menjadi organisme yang sangat kompleks.

292 folder, 5 root sistem berbeda, 3 versi orchestrator.

Ini bukan tanda kegagalan.

Ini adalah tanda bahwa SUDAH SAATNYA melakukan
KONSOLIDASI besar sebelum menambah fitur baru.

Sesuai kalibrasi MICO OMEGA:

"Jangan tambah alat baru sebelum alat yang ada terpakai maksimal."

Langkah selanjutnya bukan menambah integrasi agen baru.

Langkah selanjutnya adalah:

1. Bersihkan file sampah di root
2. Konsolidasi orchestration ke satu versi
3. Lengkapi adapter folder untuk 4 agen yang kurang
4. Kirim input Jarvis dan Perplexity untuk diaudit

...

— Chief Audit Officer | JDEQ-AUDIT-BOARD

Siap melanjutkan audit setelah input Jarvis dan Perplexity diterima.